Measuring Software Patch Impact in Open Source Projects:

A Heuristic-Based Evaluation Framework

Athanasios Karathanasis

Master's Thesis (40 ECTS)

MSc in Software Engineering

University of Southern Denmark

Supervisor:

Abhishek Tiwari

Submission Date:

1st June 2026

Acknowledgements

I would like to express my sincere gratitude to my parents and my family for their continuous support, patience, and encouragement throughout my studies. Their belief in me has been a constant source of strength during this journey.

I am also deeply thankful to my friends and fellow students for their support, discussions, and shared experiences, which made this process more manageable and meaningful.

I would like to thank my supervisor, Abhishek Tiwari, for his guidance, feedback, and support during the development of this thesis, as well as for his teaching in software engineering.

I would like to express my appreciation to Aisha Umair for her excellent teaching in Scientific Methods, which significantly shaped my understanding of research and academic writing.

I would also like to thank Mikkel Baun Kjærgaard and Sofie Birch for their support and coordination of the MSc programme in Software Engineering.

In addition, I would like to thank Torben Worm and Eun-Young Kang for their teaching in Advanced Software Architecture and Analysis, Jakob Hviid for his contribution to Big Data and Data Science Technology, and Aslak Johansen for Software Technology for Internet of Things.

Furthermore, I would like to thank Ali Muhammad, Maximus Kaos, Miguel Enrique Campusano Araya, Jan-Matthias Braun, and Mahyar Tourchi Moghaddam for their teaching and contribution across the programme.

I would also like to express my gratitude to Denmark for providing a high quality educational environment and the opportunity to pursue my studies. The historical connection between Greece and Denmark, as well as the shared values and mutual respect between the two peoples, made this experience even more meaningful.

Finally, I would like to thank all those who supported me throughout this journey, both directly and indirectly.

Abstract

Software patches are widely used to modify and improve existing systems, yet their impact on program structure and behaviour is often difficult to evaluate. This thesis presents a framework for measuring the impact of software patches in open source projects developed in C and C++.

The proposed approach combines static and dynamic software metrics to capture both structural and behavioural changes between program versions. Metrics such as code complexity, execution time, and memory usage are used to quantify differences between original and patched code.

A metric based and heuristic driven framework is implemented to analyse software patches and estimate their impact. The framework aims to identify changes that preserve intended behaviours while minimizing unnecessary complexity and to support the comparison of patch quality.

The approach is evaluated using real and synthetic patch datasets, demonstrating how measurable software metrics can support a systematic and reproducible evaluation of software changes and assist developers in understanding the impact and quality of patches.

Contents

Introduction

Software systems evolve continuously over time. As requirements change, bugs are discovered, performance limitations emerge, and new constraints appear, developers must modify existing code in order to keep systems useful, reliable, and maintainable. In practice, many of these modifications are introduced in the form of software patches. A patch may correct a defect, improve execution speed, reduce memory consumption, simplify part of an implementation, or adapt the software to new conditions. Because of this, patches are a central part of software maintenance and software evolution. At the same time, however, patches are not always easy to evaluate in a reliable and systematic way. A code modification that appears beneficial at first sight may also have side effects that are less visible, less immediate, or harder to measure directly.

This makes patch evaluation an important problem in software engineering. When a developer changes an existing program, the question is not only whether the new version compiles or passes a limited set of tests. It is also whether the change preserves intended behaviour, whether it remains minimally invasive, whether it introduces unnecessary structural complexity, whether it affects runtime behaviour or resource usage, and whether it creates new risks for the system. In software engineering practice, it is well known that an attempt to fix one bug may unintentionally introduce several new problems (Kim, Park, & Yoo, 2023, p. 314). This concern reflects a broader issue. A patch should not be judged only by its immediate visible effect, but also by the deeper structural and behavioural consequences it may cause within the software system.

In many practical settings, patch quality is assessed through a combination of testing, code review, and manual inspection (Souza, Chavez, & Bittencourt, 2015). These approaches are useful and often necessary, but they also have limitations. Testing may confirm that expected outputs are produced for selected cases, yet still fail to reveal hidden changes in program logic, altered execution paths, or newly introduced inefficiencies (Spring & Illari, 2018, p. 638). Code review can identify suspicious changes, but it often depends on human judgement, available time, and knowledge of the system (Yang, et al., 2024, p. 5). Manual inspection may help developers reason about the code, but it is difficult to scale and may not provide objective and reproducible measurements (Zia, Hanke, Skrotzki, Völker, & Bayerlein, 2024, p. 258). As a result, there is a need for methods that support patch evaluation through systematic, measurable, and repeatable analysis rather than relying only on informal judgement.

The difficulty becomes even more significant when the visible output of the program remains correct. A patched version may still produce the same result during normal use while becoming more complex, less efficient, or harder to maintain (Reis, Abreu, & Cruz, 2021, p. 1). More importantly, the internal behaviour or logic of the program may have changed in ways that are not immediately visible. A patch may alter control flow, introduce new resource costs, change timing characteristics, or create states that were not previously possible

(Xiao, Yang, Wang, & Xiong, 2024; Islam, Prokhorenko, & Babar, 2023, p. 625). In some cases, it may even introduce a security weakness, whether intentionally or unintentionally (Longo, Pintore, Rinaldo, & Sala, 2017, p. 174). These possibilities make it important to examine not only what a patch appears to achieve externally, but also how it affects the internal qualities of the software.

From an academic perspective, this problem connects several important areas of software engineering. It relates to software maintenance because patches are a basic mechanism for software evolution (Islam, Prokhorenko, & Babar, 2023, p. 111652). It relates to program analysis because meaningful evaluation requires understanding both code structure and runtime behaviour (Sobhy, Bahsoon, Minku, & Kazman, 2021; Grazia & Pradel, Code Search: A Survey of Techniques for Finding Code, 2022). It relates to software quality because patches may influence maintainability, complexity, efficiency, reliability, and stability (Ramsauer, Lohmann, & Mauerer, 2016; Reis, Abreu, & Cruz, 2021). It also relates to empirical software engineering, since the problem calls for methods based on measurable evidence, controlled analysis, and reproducible evaluation (Furtado, Vignando, Fran a, & OliveiraJr, 2019; Bogner & Verdecchia, 2025, p. 1396). These connections make software patch impact a relevant and valuable topic for a master's thesis in software engineering.

This thesis addresses the problem by investigating how software patches can be evaluated in a systematic and measurable way through the use of static and dynamic software metrics. The study focuses on open source projects written in C and C++. These languages are particularly relevant because they are widely used in systems programming and performance sensitive software, where even relatively small code changes may have important effects on behaviour, efficiency, and resource usage (Bilokon, Lucuta, & Shermer, 2023; Brunner, Pataki, & Porkoláb, 2016, p. 2). In this context, the thesis examines how measurable properties of code and execution can be used together to analyse differences between original and modified versions of software.

The central idea of the thesis is that patch evaluation should consider both structural and behavioural aspects of software change. Structural aspects concern how the source code itself is modified. These include properties such as code complexity and other characteristics that can be observed without executing the program. Behavioural aspects concern what happens when the program runs, including execution time, memory usage, and other runtime effects. A method that focuses only on structure may miss important runtime consequences, while a method that focuses only on runtime behaviour may fail to explain how the internal structure of the code has changed (Sobhy, Bahsoon, Minku, & Kazman, 2021; Lazreg, Cordy, Collet, Heymans, & Mosser, 2019). For this reason, the thesis combines static and dynamic metrics in order to provide a broader and more balanced view of patch impact.

To support this goal, the thesis proposes the design and evaluation of a tool supported framework for assessing patch impact and patch quality. The framework is intended to analyse pairs of program versions, including an original version

and a modified or patched version, and to compute differences using selected metrics. These measurements are then used within a heuristic based evaluation approach. The purpose of the heuristic is to estimate the significance of the change, the logical distance between buggy and patched versions, and the extent to which a patch preserves intended behaviour while remaining minimally invasive. The framework also aims to support comparative patch evaluation by helping distinguish between patches that appear well designed and those that may be poorly designed, overfitted, or unnecessarily invasive. In other words, the framework does not attempt only to show that a patch is different. It aims to provide a reasoned and measurable basis for evaluating how different the patch is, in what ways it differs, and whether those differences appear beneficial or problematic.

An important concept in this work is the idea of logical distance between program versions. In this thesis, this idea refers to the degree to which a patch changes the structure and behaviour of the software relative to the original version. A small and well targeted patch may correct a problem while preserving most of the original design and runtime characteristics. A more invasive patch may solve a defect, but at the cost of increased complexity, higher resource consumption, or altered behaviour. By using a combination of metrics and heuristic rules, the proposed framework seeks to estimate this form of distance in a way that is practical and useful for comparative evaluation. The broader aim is to support decisions about patch quality by moving beyond purely intuitive judgement towards an evidence based assessment process.

The empirical part of the study is based on software patches and corresponding source code versions collected from open source C and C++ projects. Whenever possible, the analysis uses paired versions that allow direct comparison between the program before and after modification. In addition to real patches, the thesis also uses synthetic patches created for experimental purposes. This combination is useful because real patches provide practical relevance, while synthetic patches make it possible to explore controlled scenarios and specific kinds of changes that may not be easy to isolate in naturally occurring datasets. Together, these materials provide the basis for evaluating the usefulness of the framework and for examining which metrics are most informative when assessing software patch impact.

The scientific contribution of the thesis lies in the design, implementation, and evaluation of a measurable and reproducible method for comparing software patches. Rather than proposing an automated repair system or generating new patches automatically, the thesis focuses on the evaluation of existing modifications. This distinction is important. The purpose is not to create patches, but to understand and assess them. In doing so, the study contributes to the broader discussion of how software changes can be analysed more systematically and how developers can be supported in reasoning about the consequences of code modification.

The objective of the thesis is therefore to investigate how static and dynamic

software metrics can be used together within a metric based and heuristic driven framework for evaluating software patch impact in C and C++ projects. More specifically, the study asks whether such an approach can quantify structural and behavioural effects, distinguish between high quality and poorly designed or overfitted patches, and assist developers in understanding the quality and consequences of software changes. This objective reflects both the practical motivation of the work and its academic focus on measurable and reproducible software engineering methods.

The thesis is guided by the following research questions:

RQ1. How can a combination of static and dynamic code metrics be used to quantify the structural and behavioural effects of software patches?

RQ2. Can heuristic based models reliably distinguish high quality patches from those that are poorly designed or overfitted?

RQ3. What types of software metrics are most effective for assessing patch impact in compiled languages such as C and C++?

RQ4. How can a metric based evaluation framework assist developers in assessing the quality and impact of software patches?

The remainder of this thesis is organised as follows. Background and Related Work presents the theoretical and research context of the study, including software patches and software maintenance, software metrics, static and dynamic program analysis, and patch evaluation methods. Methodology explains the research approach, data collection process, metric selection, analysis method, and heuristic evaluation model. Framework Design and Implementation presents the system architecture, the static and dynamic analysis modules, the heuristic evaluation engine, and the implementation details. Experimental Setup describes the dataset, the experimental environment, and the measurement procedure. Results presents the findings of the study. Analysis and Discussion interprets the results, examines their implications, and addresses the limitations of the work. Conclusion summarises the main outcomes of the thesis, while Future Work outlines possible directions for further research. The thesis ends with References and Appendices.

Background and Related Work

This chapter presents the theoretical background and related research relevant to the evaluation of software patches. It introduces the main concepts and research areas that support the work of this thesis, including software patches and software maintenance, software metrics, static and dynamic program analysis, and patch evaluation methods.

The chapter serves two main purposes. First, it establishes the conceptual foundation needed to understand why software patch evaluation is an important problem in software engineering. Second, it positions the thesis in relation to existing research and shows how the proposed framework builds on previous work while addressing a more specific goal, namely the systematic evaluation of patch impact and patch quality through a combination of static and dynamic metrics and a heuristic based assessment approach.

Software patches are closely connected to software maintenance, since maintenance activities frequently involve modifying source code after initial release in order to fix bugs, improve performance, preserve reliability, or adapt software to changing requirements (Sejfia, Zhao, & Medvidović, 2021, p. 305). At the same time, patch evaluation is also related to software quality assurance, because code changes may affect maintainability, correctness, efficiency, and other important properties of software systems (Fei, et al., 2024; Chen & Jiang, Evaluating Software Development Agents: Patch Patterns, Code Quality, and Issue Complexity in Real-World GitHub Scenarios, 2024). For this reason, the study of patches cannot be separated from broader software engineering concerns such as bug fixing, software analysis, software quality, and empirical evaluation (Kim, Park, & Yoo, 2023; Li & Paxson, 2017; Zhong & Su, 2015, p. 314).

The literature relevant to this thesis spans both conceptual and empirical work. On the conceptual side, prior work in software maintenance, software quality assurance, and software repair helps explain why patches matter and what kinds of risks and changes they may introduce (Ramsauer, Lohmann, & Mauerer, 2016; Gamage, 2022). On the empirical side, research on bug fix patterns, repeated fixes, concise patch extraction, defectiveness prediction, patch noise, and patch comparison provides useful insight into how patches can be studied and assessed in practice (Sobreira, Durieux, Madeiral, Monperrus, & Maia, 2018; Yue, Meng, & Wang, 2017; Liu, et al., 2018, p. 1).

The following sections examine these topics in more detail. Software Patches and Software Maintenance introduces the maintenance context in which patches arise and reviews research on bugfixing activity and patch behaviour in real projects. Software Metrics discusses measurable properties that can be used to assess structural and behavioural characteristics of software changes. Static and Dynamic Program Analysis explains the two main analytical perspectives used in this thesis and why they complement each other. Finally, Patch Evalua-

tion Methods reviews previous approaches for assessing patch correctness, patch quality, and related forms of software change evaluation.

2.1 Software Patches and Software Maintenance

Software patches are one of the most common and important ways in which software systems evolve after their initial release. In practice, a patch is a modification applied to existing software in order to correct a defect, improve performance, adjust behaviour, or respond to changing requirements or operating conditions (Islam & Sandborn, 2023, p. 12). Because of this, patches are closely tied to software maintenance, which concerns the modification of software after deployment in order to preserve or improve its usefulness, quality, and reliability (Gomes dos Anjos & Zenha-Rela, 2017, p. 35).

From a software engineering perspective, maintenance is not a secondary activity but a central part of the software life cycle (Lukić, 2022, p. 101). Once software is released, defects are discovered, assumptions change, new usage conditions appear, and systems must continue to function correctly under evolving circumstances. Bug fixing is therefore a major maintenance activity, and many software patches are created precisely to correct faults that become visible during real use, testing, or code review (Lukić, 2022, p. 101). This maintenance context is important for the present thesis because it shows that patch evaluation is not only a question of code difference, but also a question of how software evolves under continuous change.

Research on software repair further reinforces this view. Repair is often presented as the activity of modifying a faulty system so that it behaves correctly or more safely under a given set of expectations (Górski & Kamiński, 2016, p. 67). In simple cases, the repair may seem small, such as adding a condition, changing a synchronization point, or correcting a memory-related operation. However, even simple repairs are not necessarily straightforward. A small patch may still affect execution paths, alter state changes, or create new side effects (Islam, Prokhorenko, & Babar, 2023, p. 111681). This is one reason why software repair and software maintenance should not be understood only as local edits, but as interventions that may have broader structural and behavioural consequences.

This idea is also visible in work on the effect of a patch on program state. A patch does not merely change text in a source file. It may also change control flow, variable states, object states, and runtime behaviour (Islam, Prokhorenko, & Babar, 2023; Böhme, Soremekun, Chattopadhyay, Ugherughe, & Zeller, 2017, p. 111657). In this sense, the impact of a patch can be understood not only syntactically, but also semantically and operationally. This perspective is especially relevant to the present thesis, since one of its central goals is to estimate the logical distance between an original and a patched version of a program. The notion that patch impact may be visible through state changes, execution traces, or control flow changes provides a strong motivation for evaluating soft-

ware patches with more than one type of measurement.

Another important source of insight comes from mining software repositories. Version control systems contain a large amount of information about how developers actually fix bugs in practice. Commit histories, commit messages, code diffs, and associated metadata allow researchers to systematically study how a bug was fixed, how often similar fixes occur, and what kinds of maintenance behaviour are repeated across projects (Grazia, Bredl, & Pradel, DiffSearch: A Scalable and Precise Search Engine for Code Changes, 2022; Tufano, et al., 2019, p. 2366). This repository perspective is useful because it grounds the study of patches in real development activity rather than abstract examples alone.

Empirical studies have shown that bug fixes often follow recurring patterns (Yue, Meng, & Wang, 2017, p. 422). Research on repeated bug fixes suggests that a noticeable portion of bugs involve repeated or highly similar fixes applied to multiple code locations (Yue, Meng, & Wang, 2017, p. 422). This is important because it indicates that patching behaviour is not purely random or completely unique in every case. Instead, some classes of defects and some forms of maintenance work tend to generate recurring repair actions (Koyuncu, et al., 2020, p. 2014). Similar observations appear in studies of common bug fix patterns, where large scale empirical analysis has shown that certain patch structures or bug fixing actions occur far more frequently than others (Yue, Meng, & Wang, 2017, p. 422). Such findings are useful for the present thesis because they support the idea that patch behaviour can be studied systematically and that patch properties may be compared across versions and cases.

Research has also shown that bug fixing commits in version control systems are not always concise representations of the actual repair (Jiang, Liu, Niu, Zhang, & Hu, 2021, p. 686). A human-written patch may contain bug relevant changes together with refactoring, formatting adjustments, cleanup actions, or other modifications that are not essential to the fix itself. This is why work on extracting concise bug fixing patches is important. If a patch contains both essential and nonessential changes, then evaluating the impact of the patch becomes more difficult. A comparison between original and modified versions may otherwise capture noise in addition to the real repair effect. For this thesis, this observation is significant because it supports the need to think carefully about what part of a software patch should be measured and how one should interpret the measured difference between versions.

Maintenance research also highlights that applying a patch is not always the end of the process. In some cases, patches are backed out or reverted because they introduce new problems, destabilise the system, or fail under broader testing or integration conditions (Souza, Chávez, & Bittencourt, 2015, p. 94). Studies of rapid releases and patch backouts illustrate that software maintenance includes not only patch construction, but also patch monitoring, integration, and correction of failed changes (Souza, Chávez, & Bittencourt, 2015, p. 94). This is an important reminder that a patch should not be treated as automatically successful simply because it was proposed or committed. It must also be considered in

terms of its downstream effects on the system and the development process.

Practitioner-oriented studies provide another useful perspective. Work that examines where practitioners locate bugs and how they fix them shows that debugging and patching are not purely mechanical activities (Böhme, Soremekun, Chattopadhyay, Ugherughe, & Zeller, 2017, p. 128). Developers form hypotheses, interpret symptoms, inspect code, and gradually construct a fix (Böhme, Soremekun, Chattopadhyay, Ugherughe, & Zeller, 2017, p. 128). This helps explain why patch evaluation is difficult. A patch may appear correct at a superficial level while still failing to reflect the deeper logic of the problem or while introducing additional issues. For this thesis, such observations reinforce the need for patch evaluation methods that go beyond simple acceptance of a changed code fragment and instead examine the structural and behavioural consequences of the modification.

Taken together, these perspectives show that software patches should be understood as a central part of software maintenance practice. They are shaped by recurring bug fixing behaviour, repository history, repair strategies, and quality concerns. They may be small or large, clean or noisy, successful or later reverted. Most importantly, they represent real maintenance decisions whose impact may extend beyond the immediate bug fix. This is precisely why they are worth evaluating in a systematic and measurable way.

For the purposes of this thesis, the maintenance perspective provides an essential foundation. It shows that patch impact should be analysed within the broader context of software evolution and not merely as a superficial text difference between two files. It also supports the decision to use real and synthetic patches, to compare original and modified versions, and to examine both structural and behavioural effects when evaluating patch quality and patch impact.

2.2 Software Metrics

Software metrics provide a systemati1c way to describe, compare, and evaluate properties of software systems through measurable values (Alenezi, 2016, p. 3). In software engineering, metrics are used to capture characteristics of source code, program structure, execution behaviour, maintainability, and resource usage. Because the present thesis is concerned with evaluating the impact of software patches in a measurable and reproducible way, software metrics form one of its most important conceptual foundations.

At a general level, software metrics allow software artefacts to be studied beyond purely informal judgement. Instead of relying only on intuition, visual inspection, or limited testing, metrics provide quantitative indicators that make comparison more structured and less subjective (Birihanu, 2018, p. 11). This does not mean that metrics replace engineering judgement. Rather, they support it by offering observable values that can help reveal how a modification influences the system. In the context of patch evaluation, this is especially useful because

a patch may appear simple or correct at first sight while still producing deeper structural or behavioural consequences.

Software metrics are often divided into broad categories depending on what aspect of the system they describe. One important distinction is between structural metrics and behavioural or runtime metrics. Structural metrics focus on properties that can be derived from the source code itself without executing the program (Khleel & Nehéz, 2021, p. 5). These may include measures related to complexity, control flow, maintainability, size, or code organisation. Behavioural metrics, in contrast, concern what happens when the software is executed (Flater, et al., 2016, p. 22). These may include execution time, memory usage, resource consumption, or other observable runtime effects. This distinction is highly relevant to the present thesis, since the proposed framework aims to evaluate both structural and behavioural changes introduced by patches.

A central example of a structural metric is cyclomatic complexity. Cyclomatic complexity is commonly used as an indicator of control flow complexity because it reflects the number of independent execution paths in a program or function (Zhao, 2021, p. 25). In practical terms, a higher value often suggests that the code contains more branching logic and may be more difficult to test, understand, or maintain. In the context of software patches, a change in cyclomatic complexity may indicate that a patch has introduced additional decision points or altered the internal structure of the software in a meaningful way. For this reason, cyclomatic complexity is particularly relevant when the goal is to study whether a patch remains reasonably focused or instead introduces unnecessary structural complexity.

However, structural metrics alone are not sufficient to describe the full impact of a software patch. A patch may leave structural complexity almost unchanged while still affecting runtime behaviour in important ways. This is why behavioural and performance related metrics are also necessary (Etemadi, Sharma, Madeiral, & Monperrus, 2023). Execution time, for example, provides insight into whether a patch introduces performance costs or improvements during actual program execution (Wang, Chattopadhyay, Gotovchits, Mitra, & Roychoudhury, 2019, p. 2513). Memory usage provides another important perspective, especially in languages such as C and C++, where low level control and efficiency are often central concerns (Xiao, Yang, Wang, & Xiong, 2024, p. 630). A patch that preserves the visible output of a program may still increase runtime cost, consume additional memory, or alter the performance profile of the software. Such effects may not be clear from code inspection alone (Etemadi, Sharma, Madeiral, & Monperrus, 2023).

Another important reason for using software metrics is comparability. In this thesis, patches are not studied only as isolated code edits. They are studied as changes between versions of software. Metrics provide a basis for comparing original and patched versions in a consistent way. This supports the broader goal of estimating patch impact and logical distance between versions. If the original and modified programs can be described through comparable structural and

behavioural indicators, then it becomes possible to reason more systematically about how much a patch changes the software and whether that change appears proportionate to the purpose of the fix.

At the same time, software metrics should not be treated as perfect or self sufficient explanations (AlOmar, 2025, p. 28). A single metric rarely captures the whole meaning of a code change (Gil & Lalouche, 2016, p. 1). For example, a small increase in complexity may be justified if it prevents a serious fault, while a performance improvement may come at the cost of reduced clarity or maintainability (Karanikiotis & Symeonidis, 2024, p. 31). In addition, some software qualities are easier to measure directly than others (Setiadi, Sumitra, Karim, & Ritzkal, 2024, p. 1453). Because of this, metric based evaluation should be interpreted carefully and should combine multiple complementary indicators rather than relying on one measure alone (Liu, et al., 2020, p. 110817). This consideration is especially important in patch analysis, where the significance of a change often depends on both its structural form and its behavioural consequences (Zibaeirad & Vieira, 2024, p. 7).

Research in software engineering has repeatedly shown that metrics can be useful for evaluating properties such as complexity, maintainability, and software quality, but it has also shown that their usefulness depends on how they are selected, interpreted, and combined (Yeltsin, et al., 2025; Voas & Kuhn, 2017, p. 4). For the purposes of this thesis, software metrics are not used as isolated scores. They are used as evidence within a broader evaluation framework. Structural metrics help describe how the source code changes, while behavioural metrics help describe what effect those changes have during execution (Voas & Kuhn, 2017; Whalen, Person, Rungta, Staats, & Grijincu, 2015, p. 97). Together, they provide a measurable basis for assessing patch impact and for supporting a heuristic based comparison of patch quality.

In this sense, software metrics are not only descriptive tools but also analytical instruments. They make it possible to move from general impressions about software changes to more systematic observations about structure, behaviour, and resource usage (Yeltsin, et al., 2025; Shah, 2017, p. 4). This role makes them essential to the present study. Without measurable indicators, patch evaluation would remain largely informal. With them, it becomes possible to compare versions more rigorously, identify meaningful change, and support a more reproducible analysis of software patches.

2.3 Static and Dynamic Program Analysis

Program analysis provides a set of techniques for examining software in order to understand its structure, behaviour, and possible effects under execution (Vida, 2017; Bernardi, Francalanza, Peressotti, & Mousavi, 2025, p. 71). In software engineering, program analysis is often used to investigate how programs are written, how they behave, where defects may arise, and how modifications influence their operation (Ai, Guo, & Wong, 2020; Al-Bataineh & Moonen, 2022). Because the present thesis focuses on evaluating the impact of software patches, program analysis is a central foundation for the proposed framework.

One of the most important distinctions in this area is the difference between static and dynamic program analysis (Muench, 2019; Mastandrea, 2017, p. 22). Static analysis examines software without executing it. It focuses on the source code itself and on properties that can be derived from the program structure, such as control flow, complexity, dependencies, or other structural characteristics (Alqadi & Maletic, 2020; Kastner, Mauborgne, Wilhelm, Mallon, & Ferdinand, 2022, p. 468). Dynamic analysis, by contrast, examines software during execution (Mastandrea, 2017; Vida, 2017, p. 44). It focuses on runtime behaviour, including execution paths, timing behaviour, memory usage, and other effects that become visible only when the program is run (Shirazi, 2019; Barrera, Moraga, Polo, & Guzmán, 2019, p. 60).

Static analysis is particularly useful when the goal is to understand how a patch changes the internal structure of a program (Trautsch, Erbel, Herbold, & Grabowski, 2023, p. 1). Since it operates directly on source code, it can reveal whether a patch introduces additional branches, modifies control flow, increases structural complexity, or alters other code level properties (Karanikiotis & Symeonidis, 2024; Avery, 2017, p. 28). This is important because a patch may appear small in terms of textual difference while still changing the internal organisation of the program in a meaningful way. Static analysis therefore helps identify the structural footprint of a patch and supports the study of how code modifications affect maintainability, readability, and complexity (Karanikiotis & Symeonidis, 2024; Reis, Abreu, & Cruz, 2021, p. 28).

In the context of this thesis, static analysis is especially relevant because one of the key concerns is whether a patch remains focused and minimally invasive. If a patch solves a local problem but significantly increases structural complexity, then the change may carry maintenance costs that are not immediately visible from the external output alone (Benaroch & Lyytinen, 2022; Reis, Abreu, & Cruz, 2021). Measures such as cyclomatic complexity are useful in this setting because they help estimate how a patch affects control flow and the number of possible execution paths through the program (Wu, et al., 2024, p. 4). For this reason, static analysis provides an essential perspective on the structural impact of software changes.

At the same time, static analysis has clear limitations. It can describe what the code looks like and how it is organised, but it cannot fully describe how the

program behaves at runtime in realistic execution scenarios (Muench, 2019; Ji, Wu, Wang, Zhang, & Wang, 2018, p. 22). A patch may leave the structural form of the code almost unchanged while still affecting performance, resource usage, timing, or state behaviour (Morice, 2022; Avery, 2017, p. 75). In other cases, a patch may appear more complex structurally but actually improve runtime behaviour or reduce failure risk. This means that structural analysis alone is not sufficient for evaluating patch impact.

Dynamic analysis addresses this limitation by examining software during execution (Vida, 2017, p. 71). Through dynamic analysis, it becomes possible to observe how a patched version behaves when it runs, how long it takes to execute, how much memory it consumes, and whether its runtime behaviour differs from that of the original version (Avery, 2017, p. 73). This perspective is particularly valuable in languages such as C and C++, where efficiency, low level control, and resource usage are often important characteristics of software systems (Mann, et al., 2022). Runtime measurement can therefore reveal effects of a patch that are not visible from source code inspection alone.

Dynamic analysis is also important because some consequences of a patch emerge only under actual execution conditions. A modification may alter timing behaviour, introduce new runtime overhead, change memory access patterns, or affect execution states in ways that are difficult to infer confidently through static examination alone. In this sense, dynamic analysis provides evidence about the operational impact of a patch. It helps move the evaluation from what the code appears to do to what the software actually does when executed (Herve, 2017, p. 64).

However, dynamic analysis also has limitations of its own. The observed runtime behaviour depends on the execution environment, inputs, workload, and measurement conditions. A dynamic result may therefore capture only part of the behaviour of the program and may vary depending on how the experiment is conducted. In addition, runtime observations do not always explain why a difference occurred at the code level. Dynamic analysis may show that performance changed, but not by itself reveal which structural aspects of the patch caused the change. For this reason, dynamic analysis is valuable, but incomplete when used in isolation (Herve, 2017, p. 64).

The present thesis combines static and dynamic program analysis precisely because each approach addresses limitations in the other. Static analysis helps explain structural change, while dynamic analysis helps reveal runtime and resource related consequences. Together, they provide a broader and more balanced basis for patch evaluation (Herbold, Knieke, Rausch, & Schindler, 2025, p. 2). This combined perspective is consistent with the overall goal of the thesis, which is to assess both structural and behavioural patch impact rather than focusing on one dimension alone.

In practical terms, the integration of static and dynamic analysis supports the central idea of evaluating software patches through multiple measurable dimen-

sions. Static analysis contributes information about the shape and complexity of the code after modification. Dynamic analysis contributes information about how the modified program behaves in execution. When these perspectives are considered together, it becomes more realistic to evaluate whether a patch preserves intended behaviour, remains minimally invasive, and avoids introducing unnecessary structural or runtime costs (Herbold, Knieke, Rausch, & Schindler, 2025, p. 2).

For this reason, static and dynamic program analysis are not treated in this thesis as competing alternatives, but as complementary approaches. Their combination supports a more complete understanding of software change and provides a stronger analytical basis for the metric based and heuristic driven framework proposed in this study.

2.4 Patch Evaluation Methods

Evaluating software patches is a challenging task because a patch cannot be judged only by the fact that it changes the code or that it appears to solve an observed defect. In software engineering, patch evaluation generally concerns whether a modification is correct, whether it preserves intended behaviour, whether it introduces unnecessary complexity, and whether it creates new structural or runtime problems (Chen & Jiang, Evaluating Software Development Agents: Patch Patterns, Code Quality, and Issue Complexity in Real-World GitHub Scenarios, 2025, p. 658). For this reason, patch evaluation methods have developed across several related directions, including testing based validation, manual review, automated program repair assessment, patch defectiveness prediction, patch similarity analysis, and semantic reasoning about program change (Monperrus, 2018, p. 30).

One of the most common ways of evaluating a patch in practice is through testing (Wang, Wen, Chen, Yi, & Mao, 2019, p. 1). If a patched program compiles and passes the relevant tests, this is often treated as evidence that the patch is acceptable (Wang, Wen, Chen, Yi, & Mao, 2019, p. 1). Testing is clearly important, since it provides direct evidence about observable behaviour under selected inputs (Le-Cong, et al., 2023, p. 3). However, testing alone does not guarantee that a patch is truly high quality (Le-Cong, et al., 2023, p. 3). A patch may satisfy the available test cases while still altering hidden program logic, introducing unnecessary complexity, or causing effects that remain outside the tested scenarios (Gao, Roychoudhury, Chin, Sergey, & Rinard, 2021, p. 18). This limitation has been widely discussed in the context of automated program repair, where a patch may appear correct according to the test suite but still be considered overfitted or semantically weak (Le-Cong, et al., 2023, p. 3).

Manual review is another widely used patch evaluation method (Ye, Martínez, & Monperrus, 2021, p. 2). Developers often inspect code changes in pull requests, commits, or patch files in order to decide whether a modification is reasonable, maintainable, and consistent with the design of the system (Oliveira, et al.,

2024; Attie, Obeidat, Oh, & Yelle, 2024, p. 30). Manual review can be highly effective when experienced developers are familiar with the code base, but it also has limitations (Attie, Obeidat, Oh, & Yelle, 2024, p. 2). It is labour intensive, difficult to scale, and dependent on human judgement (Oliveira, et al., 2024; Ye, Martínez, & Monperrus, 2021, p. 30). Different reviewers may reach different conclusions about the same patch, especially when the change has subtle structural or behavioural consequences (Oliveira, et al., 2024, p. 30). For this reason, manual review remains valuable, but it is not sufficient on its own for systematic and reproducible patch evaluation (Attie, Obeidat, Oh, & Yelle, 2024; Ye, Martínez, & Monperrus, 2021, p. 2).

The field of automated program repair has become a prominent research topic, particularly concerning patch evaluation (Liu, et al., 2020, p. 110817). In this area, researchers have studied not only how patches can be generated automatically, but also how the quality of generated patches can be assessed (Liu, et al., 2020; Tian, et al., 2022, p. 110817). This has drawn attention to the distinction between patches that merely satisfy a test suite and patches that actually represent meaningful and robust repairs (Presler-Marshall, Heckman, Stolee, Bottomley, & Lynch, 2022, p. 25). Research on patch quality in automated program repair has shown that evaluation must go beyond surface correctness and consider whether a patch is unnecessarily invasive, poorly generalised, or likely to fail outside the limited validation setting (Martínez, Durieux, Sommerard, Xuan, & Monperrus, 2016; Smith, Barr, Goues, & Brun, 2015, p. 1960). This line of work is highly relevant to the present thesis because it reinforces the idea that patch evaluation requires more than a simple pass or fail judgement.

Related to this is the concept of patch overfitting (Smith, Barr, Goues, & Brun, 2015, p. 9). Overfitting occurs when a patch appears correct for the cases used in evaluation, but does not truly capture the intended behaviour of a proper repair (Martínez, Durieux, Sommerard, Xuan, & Monperrus, 2016, p. 1950). In such situations, a patch may be accepted by the available validation process while still being conceptually weak or incomplete. This problem has become one of the central challenges in patch evaluation research because it shows that visible success under limited conditions does not necessarily imply high patch quality (Liu, et al., 2020; Martínez, Durieux, Sommerard, Xuan, & Monperrus, 2016, p. 110817). The present thesis addresses a similar concern by asking whether a patch preserves intended behaviour while remaining minimally invasive and avoiding unnecessary complexity or undesirable side effects.

Another approach to patch evaluation is defectiveness prediction (Monperrus, 2018, p. 30). Rather than evaluating a patch only after full behavioural inspection, some studies attempt to predict whether a patch is likely to be defective based on measurable properties of the code change (Dong, et al., 2020). This idea is relevant because it connects patch evaluation directly to software metrics and code characteristics. If structural features, change properties, or other measurable indicators can be associated with problematic patches, then patch quality can be studied in a more systematic and comparative manner (Dong, et

al., 2020). Such work is important for the present thesis because it supports the broader argument that measurable properties of code modifications can help assess patch impact and patch quality.

Patch evaluation has also been approached through comparative and similarity based methods (Ghanbari, 2019, p. 1370). If similar bugs often lead to similar fixes, then comparing patch structure or behaviour across cases may provide useful evidence about whether a new patch looks reasonable or unusual. Studies on similar patches and repeated bug fixes suggest that patch comparison is not only possible, but also meaningful in practice (Yue, Meng, & Wang, 2017, p. 422). This is relevant to the present work because the proposed framework does not evaluate patches in isolation only. It also aims to support comparative patch evaluation by measuring differences between versions and estimating logical distance between original and patched code.

Another important line of work concerns the explanation and interpretation of patches (Girka, 2018, p. 14). Methods that aim to explain a patch or describe what a patch changes help reveal that patch evaluation is not only a matter of acceptance, but also of understanding (Liang, et al., 2019, p. 1). A patch may be easier to trust when its effect can be explained clearly in terms of structure, behaviour, or defect correction. This perspective is closely related to the present thesis, since the proposed framework seeks not merely to label patches as good or bad, but to support a more reasoned assessment based on measurable evidence.

More formal approaches to patch evaluation also exist (Li, Shezan, wei, Wang, & Tian, 2025, p. 3). Semantic reasoning about patches, including methods that reason about program behaviour through formal representations, attempts to assess whether a patch preserves or changes properties of interest in a principled way (Gao, Noller, & Roychoudhury, Program Repair, 2022, p. 3). Such methods are valuable because they move patch analysis closer to behavioural meaning rather than treating patches only as text differences. However, formal reasoning approaches may also be complex, specialised, or difficult to apply broadly in practical settings. This creates space for more lightweight but still systematic approaches that combine measurable indicators with heuristic judgement.

Taken together, existing patch evaluation methods show that the problem has no single simple solution. Testing is necessary but incomplete (Fei, et al., 2024). Manual review is valuable but subjective (Denisov-Blanch, Ciobanu, Obstbaum, & Kosiński, 2024, p. 1). Automated program repair research has shown the importance of patch quality and the risk of overfitting (Gao, Roychoudhury, Chin, Sergey, & Rinard, 2021, p. 18). Defectiveness prediction and similarity based approaches show that measurable properties of code changes can provide useful signals (Giordano, et al., 2023, p. 30). Formal reasoning methods highlight the importance of semantics, but may be difficult to apply in everyday development settings. These observations suggest that patch evaluation benefits from approaches that combine multiple perspectives rather than relying on a single criterion.

For this thesis, the most important lesson from previous work is that patch evaluation should be systematic, comparative, and evidence based. A useful patch evaluation framework should consider both structural and behavioural effects, should support comparison between original and modified versions, and should help distinguish between patches that appear well designed and those that may be poorly designed, overfitted, or unnecessarily invasive. This is the gap that the present study addresses. Instead of relying only on testing, only on manual judgement, or only on one class of metrics, the thesis proposes a metric based and heuristic driven framework that combines static and dynamic analysis in order to evaluate patch impact and patch quality in a more balanced and reproducible way.

Methodology

This chapter explains the methodological approach of the thesis and describes how the proposed framework is designed to evaluate the impact and quality of software patches. The methodology is structured around five main components: the overall research approach, the collection of empirical material, the selection of metrics, the analysis method, and the heuristic evaluation model. Together, these components provide the basis for a systematic and reproducible evaluation of software patches in open source C and C++ projects.

3.1 Research Approach

This study follows an empirical software engineering methodology with a primarily quantitative orientation. The research is based on the analysis of observable and measurable properties of software systems, particularly source code structure, program behaviour, and runtime resource usage. Rather than relying mainly on subjective judgement, the thesis evaluates software patches through measurable indicators derived from both static and dynamic analysis.

The methodological perspective of the study is motivated by the nature of the research problem. Software patches may influence structural complexity, execution behaviour, memory usage, maintainability, and other software qualities. Because these effects can be studied through measurable values, a quantitative and measurement based approach is appropriate. At the same time, the study remains empirical because it focuses on real software artefacts, namely source code, software versions, and patches collected from open source projects.

The thesis does not aim to generate patches automatically or to develop a complete automated repair system. Instead, it focuses on the evaluation of existing modifications. More specifically, it investigates how original and patched versions of software can be compared through selected metrics and how the resulting measurements can be used within a heuristic based framework for patch assessment.

The overall research design is therefore experiment driven. Original and modified versions of software are analysed, relevant static and dynamic metrics are extracted, and the results are interpreted through a comparative evaluation model. This design supports the central goal of the thesis, which is to evaluate software patch impact and patch quality in a way that is systematic, measurable, and reproducible.

3.2 Data Collection

The empirical material of this study consists of software patches and corresponding source code versions collected from open source projects written in C and C++. The main unit of analysis is a pair of related program versions, consisting of an original version and a patched version. This allows direct comparison of software before and after a modification has been introduced.

The decision to focus on open source software is motivated by both practical and scientific reasons. Open source repositories provide access to source code, version history, and patch related changes in a form that can be inspected, reproduced, and analysed systematically. This makes them well suited for empirical software engineering research, especially when the goal is to compare structural and behavioural differences across program versions.

Whenever possible, the dataset includes real patches derived from actual maintenance activity, such as bug fixes, code corrections, or performance related changes. In addition, synthetic patches may be used in order to create controlled evaluation scenarios and to explore specific kinds of software change that are difficult to isolate in naturally occurring project histories. This combination supports both practical relevance and experimental control.

The exact selection of projects and patches is guided by relevance to the thesis scope. Projects should be implemented in C or C++, sufficiently accessible for analysis, and suitable for version comparison and controlled execution. The final dataset composition may be refined during the implementation and evaluation phases, but the general data collection strategy remains focused on comparing original and patched versions of open source software through measurable structural and behavioural properties.

3.3 Metric Selection

The proposed framework relies on a combination of static and dynamic software metrics in order to capture both structural and behavioural aspects of software change. The selection of metrics is guided by the thesis objective, which is to evaluate software patch impact and patch quality in a measurable and comparative way.

A central static metric in the framework is cyclomatic complexity. Cyclomatic complexity is used because it provides an established indicator of control flow complexity and reflects the number of independent execution paths through a program or function (Alqadi & Maletic, 2020, p. 442). In the context of patch evaluation, changes in cyclomatic complexity may indicate that a patch has introduced additional branching logic or altered the structural complexity of the code. This makes it useful for assessing whether a patch remains focused and reasonably limited in scope.

Dynamic metrics are also necessary because structural metrics alone cannot fully reveal the operational consequences of a patch (Gehlot & Kaur, 2015). For this

reason, execution time and memory usage are included as core runtime metrics. Execution time is relevant because a patch may preserve visible functionality while still introducing performance costs or performance improvements (Wang, Chattopadhyay, Gotovchits, Mitra, & Roychoudhury, 2019, p. 2513). Memory usage is similarly important, especially in C and C++ software, where low level resource control and efficiency often play a significant role (Mann, et al., 2022). Together, these metrics provide insight into how a patch affects runtime behaviour and resource consumption.

In addition to these core metrics, the framework may incorporate supporting indicators derived from static analysis results, such as the presence of warnings or structural findings relevant to code quality. Such indicators are not intended to replace the main metrics, but to complement them by providing further evidence about the potential quality implications of a patch.

The selection of these metrics reflects a deliberate balance. Cyclomatic complexity captures structural change (Alqadi & Maletic, 2020, p. 442). Execution time and memory usage capture runtime effects (Couto, Pereira, Ribeiro, Rua, & Saraiva, 2017, p. 7). Supporting static findings help reveal additional code level concerns (Omri, Montag, & Sinz, 2018, p. 153). No single metric is sufficient on its own, but together they provide a broader basis for evaluating patch impact than would be possible through structure only or runtime measurement only.

3.4 Analysis Method

The analysis method of the thesis combines static analysis, dynamic analysis, and comparative version evaluation. The general workflow is based on analysing original and patched versions of software with the same set of measurements and then comparing the resulting values in order to estimate patch impact.

Static analysis is used to evaluate structural properties of the source code without executing the program (Kastner, Pister, & Ferdinand, Obtaining DO-178C Certification Credits by Static Program Analysis, 2022, p. 2). In the present study, static analysis is particularly important for measuring cyclomatic complexity and for identifying code level findings relevant to patch quality. A lightweight command line based approach is preferred so that the selected tools can be invoked programmatically from the proposed framework and integrated into an automated evaluation pipeline.

Dynamic analysis is used to evaluate runtime properties during program execution (Vida, 2017, p. 71). In this study, dynamic measurement focuses primarily on execution time and memory usage. Execution time is measured through repeated and controlled program runs, while memory usage is obtained through profiling during execution. Repeated runs are important because runtime measurements may vary depending on environmental factors and workload conditions (Laaber, Yue, & Ali, Evaluating Search-Based Software Microbenchmark Prioritization, 2024; Laaber, Basmaci, & Salza, Predicting unstable software benchmarks using static source code features, 2021, p. 1692). For this reason, dynamic results are interpreted comparatively rather than as isolated single values.

The practical implementation of the analysis method is based on a tool supported workflow. Static complexity measurement is performed through pmccabe. Static code findings are obtained through Cppcheck. Runtime execution time is measured through a custom C++ timing harness based on standard timing facilities, and memory usage is profiled through Valgrind Massif. This combination was selected because it supports C and C++, relies on free or open tools, and allows the framework to call each component externally and parse its results in a structured way.

The use of both static and dynamic analysis is methodologically important (Merlo, Ruggia, Sciolla, & Verderame, 2021; Ponta, Plate, & Sabetta, 2018, p. 102194). Static analysis helps explain how the internal structure of the code changes, while dynamic analysis reveals how the software behaves during execution (Herbold, Knieke, Rausch, & Schindler, 2025; Vida, 2017, p. 2). The combination of these two perspectives provides a more complete basis for patch evaluation than either method could provide in isolation (Latimer, 2017; Ponta, Plate, & Sabetta, 2018). This is consistent with the central aim of the thesis, which is to assess both structural and behavioural patch impact.

3.5 Heuristic Evaluation Model

The final stage of the methodology is the heuristic evaluation model. The purpose of this model is to transform metric measurements into a comparative assessment of patch impact and patch quality. Rather than relying on a single metric or a simple pass or fail outcome, the model combines several indicators in order to support a more balanced judgement.

At the individual metric level, each selected measure is interpreted in relation to the original and patched versions of the program. For example, an increase in cyclomatic complexity may suggest that a patch is structurally more invasive, while a substantial increase in execution time or memory usage may indicate that the patch introduces new runtime costs. Conversely, stable or improved values may suggest that the patch remains relatively focused and controlled. This makes it possible to assign a per metric assessment based on the observed direction and magnitude of change.

At the overall level, the framework combines these metric specific assessments into a broader patch level evaluation. The intended outcome is a grading or rating scheme that reflects the general impact and apparent quality of the patch. A patch that preserves intended behaviour, remains minimally invasive, and avoids unnecessary structural or runtime costs should receive a stronger overall evaluation than a patch that introduces substantial complexity, performance overhead, or resource related problems.

The heuristic model is intentionally designed to remain interpretable. Its purpose is not to function as a black box predictor, but to provide a transparent and reasoned way of comparing software versions. This is particularly important in the context of patch evaluation, where developers and researchers may need to understand why a patch was assessed positively or negatively.

The exact weighting, thresholds, and aggregation rules of the heuristic model may be refined during implementation and evaluation. However, the central principle remains fixed: patch assessment should be based on multiple measurable indicators and should support comparative judgement across original and modified versions. In this way, the heuristic model serves as the bridge between raw measurements and the final evaluation of patch impact and patch quality.

Framework Design and Implementation

This chapter presents the design and implementation of the PatchImpact framework developed for this thesis. PatchImpact was created as a command-line research tool for evaluating the impact of software patches in C and C++ projects. Its purpose is to support systematic comparison between original and patched software versions by collecting measurable evidence about source code structure, static code quality, runtime behaviour, and overall patch risk.

The framework is not intended to replace human judgement, code review, or testing. Instead, it provides a measurable and reproducible layer of evidence that can support patch evaluation. A developer or researcher can use the framework to inspect a single software version, compare an original version with a patched version, analyse executables, perform mass assessment across multiple artefacts, generate reports, and export results for later analysis. In this way, the framework acts both as a practical tool and as the experimental instrument used in this thesis.

The design of PatchImpact is based on the idea that patch impact should be evaluated through more than one measurement. A patch may reduce complexity but remove important functionality. Another patch may preserve the number of lines and functions but increase runtime cost. A third patch may compile and run successfully while introducing additional static analysis warnings. For this reason, PatchImpact combines static analysis, dynamic analysis, and heuristic scoring rather than relying on a single metric.

The main contribution of the framework is not the invention of each individual metric. Metrics such as cyclomatic complexity, source lines of code, runtime duration, memory usage, and static analysis warnings already exist in software engineering practice. The contribution lies in integrating these measurements into a reproducible C++ framework that compares original and patched C/C++ artefacts, calculates structural and behavioural differences, applies an interpretable heuristic model, and exports results for empirical analysis.

4.1 System Architecture

PatchImpact was implemented as a modular command-line framework in C++17. The architecture follows a pipeline model in which software artefacts are collected, analysed, compared, scored, and exported. The main unit of analysis is a pair of related artefacts, usually an original version and a patched version. The framework can also analyse single artefacts when the purpose is inspection rather than patch comparison.

The general architecture can be described as follows:

Input artefacts
↓
File and executable discovery

↓
Static analysis and dynamic analysis
↓
Metric extraction
↓
Original versus patched comparison
↓
Heuristic scoring
↓
Terminal report and CSV export

This structure separates measurement from interpretation. The static and dynamic analysis modules are responsible for collecting measurable values. The heuristic evaluation engine is responsible for interpreting the differences between those values. This separation is important because it makes the framework easier to understand, reproduce, and extend. A new metric can be added to the analysis stage without changing the entire scoring system, and the scoring model can be adjusted without changing how the raw measurements are collected.

The architecture contains five main layers.

The first layer is the input and collection layer. This layer receives command-line arguments, validates paths, identifies whether the input is a file, folder, or executable, and prepares the artefacts for analysis. For source analysis, it recursively scans folders and collects supported C and C++ files. For executable analysis, it checks whether the selected artefacts can be executed and measured.

The second layer is the static analysis layer. This layer evaluates source code without executing it. It collects metrics such as cyclomatic complexity, function count, source lines of code, comment lines, blank lines, file size, similarity, and static analysis findings from tools such as Cppcheck.

The third layer is the dynamic analysis layer. This layer evaluates compiled executables during execution. It records runtime-related metrics such as elapsed time, user CPU time, system CPU time, total CPU time, maximum memory usage, binary size, and exit status.

The fourth layer is the heuristic evaluation layer. This layer converts raw metric differences into partial impact scores. Structural metrics contribute to the structural impact score. Runtime metrics contribute to the behavioural impact score. Static analysis findings contribute to the quality impact score. These partial scores are then combined into an overall patch impact score, grade, and risk interpretation.

The fifth layer is the reporting and export layer. This layer prints human-readable reports to the terminal and can also write the same output to report files. It can also export structured CSV data for later analysis in spreadsheet or statistical tools. This is important for the thesis because it allows raw framework results to be preserved and reused in the results chapter and appendices.

PatchImpact supports several execution modes. These modes were implemented so that the same framework can be used both for small manual experiments and for larger dataset-based analysis.

Single folder source analysis scans one source folder and reports metrics for supported C and C++ source files:

./patchimpact_v41 <source_folder>

Folder comparison mode compares an original folder against a patched folder:

./patchimpact_v41 <original_folder> <patched_folder>

This is the main patch evaluation mode. It compares matching source files, detects structural differences, computes static metric changes, and produces structural, quality, and overall patch assessment results.

Build analysis mode compares whether two project folders build successfully:

./patchimpact_v41 -ba <original_folder> <patched_folder>

or:

./patchimpact_v41 --build-analysis <original_folder> <patched_folder>

Dynamic measurement mode compares two executables:

./patchimpact_v41 -dm <original_executable> <patched_executable>

or:

./patchimpact_v41 --dynamic-measurements <original_executable> <patched_executable>

Individual assessment mode inspects one source file, executable, or folder:

./patchimpact_v41 -ia <folder_or_file>

or:

./patchimpact_v41 --individual-assessment <folder_or_file>

This mode is useful when the purpose is to inspect a single artefact rather than evaluate a patch pair. It reports static metrics for source files and dynamic or binary metrics for executables.

Mass assessment mode performs pairwise directional comparison of compatible artefacts inside a folder:

./patchimpact_v41 -ma <folder>

or:

./patchimpact_v41 --mass-assessment <folder>

This mode is useful for exploratory experiments because it evaluates multiple possible comparisons automatically. The comparison is directional. A change from artefact A to artefact B is not treated as equivalent to a change from

artefact B to artefact A, because patch impact depends on which version is considered the original and which version is considered the patched version.

PatchImpact also supports report capture:

./patchimpact_v41 -rep <report_file> <existing_command>

and CSV export:

./patchimpact_v41 -csv <output_file> <existing_command>

For example:

./patchimpact_v41 -csv mass_results.csv -ma experiment_folder

The command-line design was chosen deliberately. A graphical interface was not necessary for the research objective. A command-line tool is easier to combine with shell scripts, dataset folders, build systems, and automated experiment workflows. It also improves reproducibility because each experiment can be described by the exact command used to generate the results.

4.2 Static Analysis Module

The static analysis module evaluates source code without executing the program. Its purpose is to measure how a patch changes the structure and static quality of the software. This is important because a patch can affect maintainability, control flow, warning counts, and code size even when the program still compiles and produces the expected output.

The static analysis module works on individual source files and on folders containing multiple C and C++ files. The SourceFileCollector component recursively traverses the selected folder and identifies supported file types such as .c, .cpp, .h, and .hpp. Unsupported files are ignored so that the framework remains focused on the C and C++ artefacts relevant to the thesis.

The static analysis module collects several metrics.

Cyclomatic complexity is collected using pmccabe. This metric estimates the number of independent control-flow paths through the program or function. In the context of patch evaluation, an increase in cyclomatic complexity may indicate that the patch introduces additional branching, conditional logic, or execution paths. A decrease may indicate simplification, but it must be interpreted carefully because complexity may also decrease when functionality is removed.

Function count is collected to identify whether a patch adds, removes, or changes the number of functional units in the analysed code. A change in function count can be meaningful because adding functions may indicate new functionality or decomposition, while removing functions may indicate simplification, deletion, or possible loss of behaviour.

Source Lines of Code, or SLOC, are counted by the LineAnalyzer. In PatchImpact, SLOC is used as an indicator of the amount of source code that con-

tributes to the implementation. It helps distinguish small local patches from larger changes that modify or introduce more code. SLOC is not interpreted as quality by itself, but as one structural signal among several.

Comment lines are counted separately from source lines. Comments do not directly affect runtime behaviour, but they provide information about documentation density and maintainability context. A patch that changes only comments should not be treated as structurally equivalent to a patch that changes executable logic.

Blank lines are also counted. Blank lines mainly provide formatting context. They help separate layout or formatting changes from substantive code changes. A patch that changes only blank lines should produce low structural impact.

Static code quality findings are collected using Cppcheck. The framework runs Cppcheck on analysed source files and counts relevant findings such as errors, warnings, style issues, performance suggestions, portability concerns, and informational messages. These findings provide a code quality signal. A patch that introduces new warnings may require closer inspection, while a patch that removes warnings may indicate improvement. However, Cppcheck output is not treated as proof of correctness or incorrectness. It is interpreted as one indicator within the overall heuristic model.

For pairwise comparison, PatchImpact stores the original value and patched value for each metric. It then calculates the absolute difference:

metric_change = patched_value - original_value

When meaningful, it also calculates a percentage change:

percentage_change = ((patched_value - original_value) / original_value) × 100

If the original value is zero, the framework must avoid division by zero. In such cases, the change is treated as a special case rather than a normal percentage. For example, a change from zero functions to five functions is not simply a percentage increase. It represents a structural introduction of functionality. Similarly, a change from nonzero functions to zero functions may indicate deletion or removal of behaviour and should be interpreted cautiously.

The static analysis process can be summarised as follows:

Collect source files
↓
Run complexity analysis
↓
Count lines, comments, blanks, and functions
↓
Run static quality analysis
↓
Compare original and patched values

↓
Produce structural and quality evidence

A simplified representation of the source file collection logic is shown in Listing 4.1.

// Listing 4.1: Simplified source file collection logic

```
for (const auto& entry : std::filesystem::recursive_directory_iterator(rootPath))
{
if (!entry.is_regular_file()) {
continue;
}

std::string extension = entry.path().extension().string();

if (extension == ".c" ||
extension == ".cpp" ||
extension == ".h" ||
extension == ".hpp") {
sourceFiles.push_back(entry.path());
}
}
```

This listing illustrates the role of the collection layer. The framework does not assume that every file in a project should be analysed. Instead, it filters the project and extracts the artefacts relevant to C and C++ patch evaluation.

The static analysis module therefore creates measurable evidence about the structural footprint of a patch. It cannot prove semantic correctness, but it can show whether a patch changes code size, control-flow complexity, function structure, documentation context, and static warning profile. These values form the basis for structural and quality scoring in the heuristic evaluation engine.

4.3 Dynamic Analysis Module

The dynamic analysis module evaluates executable behaviour. While the static analysis module studies source code without running it, the dynamic analysis module measures what happens when compiled artefacts are executed. This distinction is important because a patch may produce small source-level changes but still affect execution time, memory use, binary size, or program termination behaviour.

The dynamic module is used primarily through dynamic measurement mode:

./patchimpact_v41 -dm <original_executable> <patched_executable>

The original executable and patched executable are executed under the same measurement procedure. PatchImpact collects runtime metrics and compares

the observed values. The framework uses /usr/bin/time -v and internal timing logic to record execution behaviour.

The dynamic metrics collected by the framework include elapsed execution time, user CPU time, system CPU time, total CPU time, peak memory usage, binary size, and exit status.

Elapsed execution time represents the wall-clock time required for the executable to complete. It is useful because it describes the time observed by the user or external process. A patched executable with much higher elapsed time may indicate a performance cost.

User CPU time measures the amount of CPU time spent executing user-level code. This helps identify whether a patch increases computational work inside the program itself.

System CPU time measures the amount of CPU time spent in operating system or kernel-level operations on behalf of the process. This can be relevant when a patch changes file operations, process interaction, memory allocation behaviour, or other system-level activity.

Total CPU time is calculated as the combination of user CPU time and system CPU time:

total_cpu_time = user_cpu_time + system_cpu_time

Peak memory usage records the maximum resident memory used during execution. In C and C++ programs, memory behaviour is especially important because these languages often expose low-level resource management risks. A patch that substantially increases memory usage may introduce a resource cost even if it preserves visible functionality.

Binary size is measured to identify whether the compiled artefact grows or shrinks after the patch. Binary size is not a direct measure of correctness, but it can help indicate whether the patch introduces additional compiled code or dependencies.

Exit status records whether the program terminated successfully or failed. This metric is essential because a patch that produces abnormal termination should be treated as risky, even if other metric values appear favourable.

For each dynamic metric, PatchImpact calculates absolute and relative changes between the original and patched executables. For example:

elapsed_change = patched_elapsed - original_elapsed

memory_change = patched_memory - original_memory

binary_size_change = patched_binary_size - original_binary_size

For relative change:

relative_change = ((patched_value - original_value) / original_value) × 100

As with static metrics, the framework must handle zero or missing values carefully. A failed execution, unavailable measurement, or zero baseline is not treated as a normal numeric comparison. Instead, such cases are interpreted as special conditions that may increase risk or reduce confidence in the result.

The dynamic analysis process can be summarised as follows:

Receive original and patched executables
↓
Validate executable paths
↓
Measure original executable
↓
Measure patched executable
↓
Compare timing, CPU, memory, binary size, and exit status
↓
Produce behavioural impact evidence

A simplified representation of dynamic measurement is shown in Listing 4.2.

```
// Listing 4.2: Simplified dynamic measurement workflow

DynamicMetrics originalMetrics = measureExecutable(originalExecutable);
DynamicMetrics patchedMetrics = measureExecutable(patchedExecutable);

DynamicComparison comparison;
comparison.elapsedChange =
patchedMetrics.elapsedSeconds - originalMetrics.elapsedSeconds;

comparison.memoryChange =
patchedMetrics.maximumResidentMemoryKb -
originalMetrics.maximumResidentMemoryKb;

comparison.binarySizeChange =
patchedMetrics.binarySizeBytes - originalMetrics.binarySizeBytes;

comparison.exitStatusChanged =
patchedMetrics.exitStatus != originalMetrics.exitStatus;
```

Dynamic analysis is useful because it reveals behavioural effects that static analysis alone cannot capture. A patch may leave complexity unchanged but increase memory use. It may reduce lines of code but slow down execution. It may improve performance but change exit behaviour. These cases show why dynamic metrics must be interpreted together with static and quality metrics rather than separately.

At the same time, dynamic metrics have limitations. Runtime behaviour depends on hardware, operating system state, compiler flags, input data, and

background system load. For this reason, PatchImpact treats dynamic measurements as comparative evidence rather than absolute truth. The most important question is not whether one execution time is universally good or bad, but whether the patched artefact behaves differently from the original artefact under the same measurement procedure.

4.4 Heuristic Evaluation Engine

The heuristic evaluation engine is the interpretation component of PatchImpact. Its purpose is to transform the raw static, dynamic, and quality measurements collected by the framework into patch-level impact scores, grades, and risk categories. The model is heuristic rather than formally predictive. It does not claim to prove semantic correctness. Instead, it provides an explicit and repeatable way of estimating how strongly a patch affects the analysed artefacts.

The framework uses separate scoring models for structural impact, behavioural impact, quality impact, and overall impact. Each score is normalised to the range 0 to 100, where a higher score represents lower observed impact or lower risk, and a lower score represents higher observed impact or higher risk. This means that a score close to 100 indicates that the patched version remains relatively close to the original version according to the measured indicators, while a lower score indicates greater structural, behavioural, or quality-related change.

4.4.1 Maintainability Score

Before calculating the quality impact score, PatchImpact calculates a simple maintainability score for each analysed source file. This score is not intended to represent a universal maintainability index. Instead, it is a lightweight prototype metric used inside the framework to combine several static signals into one interpretable value.

The maintainability score starts from 100 and is adjusted according to complexity, source lines of code, Cppcheck findings, and comment lines:

maintainability =

100

- (complexity $\times$ 0.5)

- (SLOC $\times$ 0.02)

- (cppcheck_findings $\times$ 2.0)

+ (comment_lines $\times$ 0.10)

The score is then clamped to the range 0 to 100:

if maintainability < 0, maintainability $= 0$

if maintainability > 100, maintainability $= 100$

The interpretation of this formula is straightforward. Higher complexity, larger source code size, and more Cppcheck findings reduce maintainability, while comment lines slightly improve the score because they may indicate additional documentation. The weighting values are heuristic and were selected to produce an interpretable prototype score suitable for comparative patch evaluation.

For a patch comparison, the framework calculates maintainability for both the original and patched versions and then computes:

maintainability_change =

patched_maintainability - original_maintainability

A negative value indicates that the patched version has lower maintainability according to the framework's metric. A positive value indicates that the patched version has improved maintainability according to the same metric.

4.4.2 Structural Impact Score

The structural impact score measures how much the source structure changes between the original and patched versions. It uses four indicators:

complexity_change_percent

function_change_percent

SLOC_change_percent

Levenshtein_similarity_percent

The percentage changes are calculated only when the original value is greater than zero. This avoids division by zero and prevents meaningless percentage calculations when no baseline exists.

The percentage change formula is:

change_percent =

((patched_value - original_value) / original_value) × 100

PatchImpact calculates this for complexity, function count, and SLOC. It also calculates textual similarity using Levenshtein distance. The Levenshtein distance is computed over the contents of the original and patched files. The similarity percentage is then derived from the distance relative to the maximum file length:

similarity_percent =

100 - ((levenshtein_distance / max_file_length) × 100)

If both files are empty, similarity is treated as 100%.

The structural score starts from 100 and subtracts penalties for structural growth and textual dissimilarity:

structural_score =

100

- positive_complexity_change_percent × 0.4

- positive_function_change_percent × 0.2

- positive_SLOC_change_percent × 0.2

- (100 - similarity_percent) × 0.2

Only positive increases in complexity, function count, and SLOC are penalised directly. This design reflects the assumption that increases in these values may indicate greater structural impact or patch invasiveness. Reductions are not automatically rewarded because a lower complexity or lower SLOC value may sometimes result from deleting or bypassing behaviour rather than improving the implementation.

The Levenshtein component penalises textual dissimilarity. A patch that changes a large portion of the source file receives a larger penalty than a patch that keeps the original and patched versions textually similar.

The structural score is also clamped to the range 0 to 100:

if structural_score < 0, structural_score $= 0$

if structural_score > 100, structural_score $= 100$

This score therefore estimates the structural footprint of a patch. It does not prove whether a patch is correct, but it indicates how much the patch changes the source artefact according to complexity, size, function structure, and textual similarity.

4.4.3 Behavioural Impact Score

The behavioural impact score is calculated for executable comparisons. This score measures runtime and binary-level changes between an original executable and a patched executable.

PatchImpact collects the following dynamic measurements:

elapsed execution time

user CPU time

system CPU time

peak memory usage

binary size

exit status

The total CPU time is calculated as:

total_CPU_time = user_CPU_time + system_CPU_time

For elapsed time, CPU time, memory usage, and binary size, PatchImpact calculates percentage change as:

change_percent =

((patched_value - original_value) / original_value) × 100

If the original value is zero or negative, the percentage change is treated as 0. This prevents division by zero and avoids producing misleading percentage values when no valid baseline exists.

The behavioural score starts from 100 and subtracts penalties for runtime or binary increases:

behavioural_score =

100

- positive_elapsed_change_percent × 0.30

- positive_CPU_change_percent × 0.30

- positive_memory_change_percent × 0.20

- positive_binary_size_change_percent × 0.10

If the exit status changes between the original and patched executable, an additional penalty is applied:

if exit_status_changed:

behavioural_score = behavioural_score - 30

The behavioural score is then clamped to the range 0 to 100.

This design gives the strongest dynamic penalties to elapsed time and CPU time, because they represent direct execution cost. Memory usage also receives a substantial penalty because memory behaviour is important in C and C++ programs. Binary size receives a smaller penalty because binary growth may indicate additional compiled code, but it is not by itself as strong a behavioural risk as runtime slowdown, CPU increase, memory increase, or changed exit behaviour.

The behavioural score is mapped to a grade and risk level using the same threshold structure as the overall model:

A: score >= 90, Low risk

B: score >= 80, Low risk

C: score >= 70, Moderate risk

D: score >= 60, High risk

F: score < 60, Critical risk

In the current implementation, behavioural scoring is fully implemented for dynamic executable comparison. However, folder comparison reports behavioural impact as not measured unless the dynamic mode is explicitly run on executable artefacts.

4.4.4 Quality Impact Score

The quality impact score measures how the patch affects static quality indicators. It uses three main inputs:

maintainability_change

cppcheck_change

build status change

The Cppcheck change is calculated as:

cppcheck_change =

patched_cppcheck_total - original_cppcheck_total

where the total Cppcheck findings are calculated as:

cppcheck_total =

errors + warnings + style + information

The quality score starts from 100:

quality_score = 100

If maintainability decreases, the score is reduced:

if maintainability_change < 0:

quality_score = quality_score + maintainability_change $\times$ 2.0

Because maintainability_change is negative in this case, adding it multiplied by 2 reduces the score. For example, a maintainability change of -5 reduces the quality score by 10 points.

If the number of Cppcheck findings increases, the score is also reduced:

if cppcheck_change > 0:

quality_score = quality_score - cppcheck_change $\times$ 5.0

This means that each additional Cppcheck finding reduces the quality score by 5 points.

The model also includes build-status penalties:

if patched_build_failed:

quality_score = quality_score - 40

else if build_status_changed:

quality_score = quality_score - 20

A patched build failure is treated as a stronger quality problem than a general build status change. The quality score is then clamped to the range 0 to 100.

In the current folder comparison implementation, the quality calculator is called with build status values set to successful for both original and patched versions. Therefore, the folder comparison quality score primarily reflects maintainability change and Cppcheck change. Build analysis exists as a separate mode and can be used to inspect whether project folders contain recognised build files, but the current overall folder score does not yet integrate full build execution.

4.4.5 Overall Patch Impact Score

*-The overall patch impact score provides the final summary assessment for source-folder patch comparison. In the current implementation, PatchImpact calculates this score by combining the structural impact score and the quality impact score with equal weighting:

overall_score =

(structural_score × 0.50)

+ (quality_score × 0.50)

This means that the source-folder comparison workflow evaluates patch impact using two main forms of evidence. The structural score represents changes in the source artefacts, including complexity change, function count change, SLOC change, and textual similarity. The quality score represents changes in maintainability, Cppcheck findings, and build-related indicators. Together, these two scores provide a static and quality-oriented assessment of the patch.

The behavioural score is not included in the overall score for source-folder comparison in the current implementation. This is a deliberate practical separation rather than a removal of dynamic analysis from the framework. Source-folder comparison operates directly on source files and can therefore be applied even when projects cannot easily be built or executed. By contrast, behavioural analysis requires runnable executable artefacts and a controlled execution command.

PatchImpact therefore treats source-level patch comparison and executable-level behavioural comparison as related but distinct workflows. The source-folder workflow produces the overall patch score using structural and quality evidence. The executable workflow produces a separate behavioural score using dynamic measurements. This separation is important because open-source C and C++ projects often use different build systems, dependencies, inputs, and runtime requirements. Automatically building and executing every patch pair is not always reliable or practical within a general research prototype.

Behavioural analysis is still implemented in the framework. It is available through dynamic measurement mode and through executable-pair assessment in mass assessment mode. In those workflows, PatchImpact compares original and patched executables using elapsed execution time, user CPU time, system CPU time, total CPU time, peak memory usage, binary size, and exit status. The dynamic workflow then calculates a behavioural score and assigns its own grade and risk level. This allows runtime behaviour to be evaluated when comparable executables are available, without making source-folder comparison dependent on successful compilation and execution.

The overall source-folder score is mapped to grades and risk levels using the following threshold model:

A: overall_score >= 90, Low risk

B: overall_score >= 80, Low risk

C: overall_score >= 70, Moderate risk

D: overall_score >= 60, High risk

F: overall_score < 60, Critical risk

The meaning of the grade is therefore metric-based rather than absolute. A grade A patch has low observed structural and quality disruption according to the implemented heuristic. A grade B patch also remains low risk but shows more measurable change. A grade C patch indicates moderate impact and may require closer inspection. A grade D patch indicates high impact. A grade F patch indicates critical impact or substantial measured risk.

This grade should not be interpreted as proof of correctness. A patch with a high grade may still be semantically wrong if it preserves the measured structure but changes intended behaviour in a hidden way. Similarly, a patch with a low grade may still be correct if the fix necessarily requires substantial restructuring or additional checks. The purpose of the grade is to summarise the observed metric-based impact and to support further human interpretation.

The overall impact score therefore acts as a compact indicator of source-level patch disruption. It combines structural evidence and static quality evidence into a single reproducible assessment, while leaving behavioural impact to the executable-oriented workflow. This design keeps the framework practical for heterogeneous C and C++ source datasets while still supporting dynamic analysis when executable artefacts are available.

4.4.6 Interpretation of Edge Cases

The heuristic model is intentionally conservative in how it interprets improvements and reductions. This is important because some metric improvements can be misleading.

A reduction in complexity is not automatically rewarded because code may become simpler by deleting functionality, bypassing error handling, or removing important behaviour. Similarly, a reduction in SLOC is not automatically treated as an improvement because fewer lines can mean either a cleaner implementation or a removed feature.

A patch that only changes comments or blank lines should normally have low structural and behavioural impact because executable logic is not substantially changed. However, such a patch may still affect maintainability context through comment counts.

A patch that introduces new Cppcheck warnings is treated as more risky because it increases static quality findings. A patch that reduces Cppcheck warnings may improve the quality score, but this improvement should still be interpreted together with structural changes.

A patch that changes exit status is strongly penalised in behavioural scoring because a change in termination behaviour may indicate that the patched executable no longer behaves like the original under the same measurement procedure.

A patch that compiles or executes faster because it skips behaviour cannot be identified as correct by the metric model alone. This is why the framework does not claim to prove correctness. Instead, it highlights measurable differences that can guide further inspection.

In summary, the heuristic evaluation engine provides an explicit and reproducible scoring model for patch impact. It combines structural evidence, quality evidence, and, where applicable, behavioural evidence. Its main value is transparency: the final score can be traced back to specific metric changes, making the patch assessment easier to inspect, explain, and compare across patch pairs.

4.5 Implementation Details

PatchImpact was implemented in C++17 using a modular object-oriented design. The framework was developed and tested on Arch Linux using GCC, CMake, and Ninja. The implementation also uses external command-line tools including pmccabe, Cppcheck, GNU time, and diff utilities. Some metrics are computed directly inside the framework, while others are collected by invoking external tools and parsing their output.

The implementation is organised around specialised classes. Each class has a clear responsibility in the pipeline. This design makes the framework easier to test, extend, and maintain.

The PatchImpactApp component coordinates the command-line interface. It receives the user arguments, determines which execution mode is requested, validates the input, and dispatches the analysis to the appropriate component.

The SourceFileCollector component discovers C and C++ source files. It recursively scans folders and filters files according to supported extensions.

The ComplexityAnalyzer component interfaces with pmccabe and extracts complexity-related metrics. It converts tool output into structured values that can be used by the rest of the framework.

The LineAnalyzer component counts source lines, comment lines, and blank lines. These metrics are used to estimate source size, documentation context, and formatting changes.

The CppcheckAnalyzer component executes Cppcheck and parses its findings into countable categories such as errors, warnings, style issues, performance issues, portability issues, and informational messages.

The DynamicMeasurementsAnalyzer component measures executable behaviour. It collects elapsed time, user CPU time, system CPU time, memory usage, binary size, and exit status.

The StructuralImpactCalculator component computes structural impact from static metric differences.

The BehavioralImpactCalculator component computes behavioural impact from runtime metric differences.

The QualityImpactCalculator component computes quality impact from static analysis finding differences.

The OverallImpactCalculator component combines partial scores into an overall patch assessment.

The IndividualAssessmentAnalyzer component supports inspection of single files, executables, or folders.

The MassAssessmentAnalyzer component supports large-scale directional pairwise comparison of compatible artefacts.

The ReportCapture component records terminal output into report files.

The CsvExportManager component writes structured CSV files for later analysis.

A simplified representation of the command dispatch logic is shown in Listing 4.4.

// Listing 4.4: Simplified command mode dispatch

```
int PatchImpactApp::run(int argc, char* argv[]) {
CommandLineOptions options = parseArguments(argc, argv);
```

```
if (options.mode == Mode::IndividualAssessment) {
return individualAssessmentAnalyzer.run(options.inputPath);
}

if (options.mode == Mode::MassAssessment) {
return massAssessmentAnalyzer.run(options.inputPath);
}

if (options.mode == Mode::DynamicMeasurement) {
return dynamicAnalyzer.compare(
options.originalExecutable,
options.patchedExecutable
);
}

if (options.mode == Mode::BuildAnalysis) {
return buildAnalyzer.compare(
options.originalFolder,
options.patchedFolder
);
}

if (options.mode == Mode::FolderComparison) {
return compareFolders(
options.originalFolder,
options.patchedFolder
);
}

return showHelp();
}
```

This simplified listing shows the role of PatchImpactApp as the entry point of the framework. The purpose of this structure is to keep command-line handling separate from the analysis logic. This makes the tool easier to extend because new modes can be added without rewriting the existing analysis components.

A simplified metric comparison operation is shown in Listing 4.5.

```
// Listing 4.5: Simplified metric comparison

MetricChange calculateChange(double originalValue, double patchedValue) {
MetricChange change;
change.originalValue = originalValue;
change.patchedValue = patchedValue;
change.absoluteChange = patchedValue - originalValue;
```

```
if (originalValue != 0.0) {
change.percentageChange =
((patchedValue - originalValue) / originalValue) * 100.0;
change.percentageAvailable = true;
} else {
change.percentageChange = 0.0;
change.percentageAvailable = false;
}

return change;
}
```

This listing demonstrates one of the most important implementation details of the framework. PatchImpact does not only collect original and patched values. It also converts them into comparative values that describe the direction and magnitude of change. This is what allows the framework to move from measurement to patch impact evaluation.

A simplified CSV export operation is shown in Listing 4.6.

```
// Listing 4.6: Simplified CSV export logic

void CsvExportManager::writeSourcePairRow(
const SourcePairResult& result
) {
csv « result.originalPath « ","
« result.patchedPath « ","
« result.originalComplexity « ","
« result.patchedComplexity « ","
« result.complexityChange « ","
« result.originalFunctions « ","
« result.patchedFunctions « ","
« result.functionChange « ","
« result.originalSloc « ","
« result.patchedSloc « ","
« result.slocChange « ","
« result.similarityPercent « ","
« result.structuralScore « ","
« result.qualityScore « "\n";
}
```

The CSV export feature is important because it separates raw experimental data from the written interpretation in the thesis. Terminal output is useful for immediate inspection, but CSV output allows results to be filtered, sorted, aggregated, and copied into thesis tables. This makes the framework more suitable for empirical evaluation.

The implementation was designed as a research prototype rather than a production system. This means that its purpose is to support controlled experiments for this thesis. The framework prioritises transparency, reproducibility, and interpretability over industrial completeness. It does not attempt to prove full semantic correctness, automatically repair software, or replace project-specific test suites. Instead, it provides a structured method for measuring and comparing patch impact.

The implementation also reflects the limitations of metric-based analysis. Static metrics cannot fully capture meaning. Dynamic metrics depend on the execution environment. External tools may report imperfect or incomplete results. The heuristic model depends on selected thresholds and weighting decisions. However, these limitations do not remove the value of the framework. They define the scope within which its results should be interpreted.

In summary, PatchImpact implements the methodology of this thesis as a working C++ tool. It collects static and dynamic measurements, compares original and patched versions, calculates metric differences, applies heuristic scoring, assigns grades and risk levels, and exports structured data for analysis. The framework therefore provides the practical foundation for the experimental evaluation presented in the following chapters.

Experimental Setup

This chapter describes the experimental setup used to evaluate the PatchImpact framework. The purpose of the experimental setup is to show how the framework was applied to real C and C++ software artefacts, how the dataset was prepared, what environment was used, and how the measurements were collected. The evaluation focuses on reproducibility, consistency, and the ability of the framework to generate measurable evidence about patch impact.

The experiment was designed around two complementary forms of analysis. First, folder comparison mode was used to compare buggy and fixed versions of selected projects. This mode represents the normal patch evaluation scenario, where one version is treated as the original or buggy version and the other is treated as the patched version. Second, mass assessment mode was used to evaluate multiple compatible artefacts inside selected folders and to export structured results to CSV format. This allowed the framework to be tested not only on individual patch pairs, but also on a larger set of pairwise source-code and executable comparisons.

The main goal of this chapter is not to prove that every patch in the dataset is correct. Instead, the goal is to evaluate whether the framework can collect and organise useful patch-impact measurements in a systematic way. These measurements are later used in the results and discussion chapters to examine structural impact, quality impact, dynamic behaviour where available, and the usefulness of the heuristic scoring model.

5.1 Dataset Description

The dataset used in this thesis was derived from open-source C and C++ projects obtained through the BugsCpp benchmark. BugsCpp provides buggy and fixed versions of real software defects, making it suitable for evaluating a framework that compares original and patched software versions. Each bug-fix instance is represented by two related program versions: a buggy version and a corresponding fixed version.

At the time of the experiment, the local dataset contained 108 buggy folders and 107 fixed folders. This corresponds to up to 107 available bug-fix pairs across nine open-source projects. The projects included both C and C++ codebases, allowing the framework to be evaluated on different implementation styles and project structures.

The downloaded projects are shown in Table 5.1. The dataset contains projects of different sizes and complexity. Smaller projects such as Berry and cpp-peglib were useful for validating the framework quickly and checking whether report generation, CSV export, and scoring behaved correctly. Larger projects such as OpenSSL, Cppcheck, Exiv2, and Coreutils were useful for testing the framework on more realistic software systems with many source files, headers, and internal components.

Table 5.1 Dataset overview

Project	Available buggy versions
berry	5
coreutils	2
cppcheck	30
cpp_peglib	1
exiv2	20
jerryscript	11
libxml2	1
md4c	10
openssl	28
Total	108

For the folder comparison experiments, each pair was treated as a direct comparison between a buggy version and its fixed version. For example, berry/buggy-1 was compared with berry/fixed-1. This allowed the framework to evaluate patch impact at project level by scanning supported C and C++ artefacts, comparing matching files by relative path, detecting changed and unchanged files, and calculating structural and quality scores.

For the mass assessment experiment, a smaller prepared dataset folder was used. This avoided running exhaustive pairwise assessment on very large project roots, where thousands of files could make the process unnecessarily expensive. The mass assessment dataset contained selected source and executable artefacts and was used to test whether the framework could generate large CSV-based results. The resulting CSV file contained 28,436 rows, consisting of 28,056 source-pair comparisons and 380 executable-pair comparisons.

It is important to clarify that the mass assessment rows do not represent 28,436 independent software patches. Instead, they represent pairwise directional comparisons between compatible artefacts discovered inside the selected dataset folder. This is consistent with the purpose of mass assessment mode, which is exploratory and intended to generate many possible comparisons for metric analysis. The actual bug-fix dataset remains based on paired buggy and fixed project versions.

5.2 Experimental Environment

The experiments were carried out on a Linux-based development environment. The primary operating system used for the evaluation was Arch Linux. This environment was selected because it provides strong support for command-line development, C and C++ compilation, static analysis tools, scripting, and reproducible software experiments.

The PatchImpact framework was built as a C++17 CMake project. The implementation was compiled and executed from the command line. This was important for reproducibility because each experiment could be described using explicit commands and file paths.

The main environment used for the experiments is summarised in Table 5.2.

Table 5.2 Experimental environment

Component	Environment detail
Operating system	Arch Linux
Desktop environment	XFCE
Programming language	C++17
Build system	CMake with Ninja
Main compiler toolchain	GCC / G++
Static analysis tool	Cppcheck
Complexity tool	pmccabe
Timing / runtime measurement	Internal timing and /usr/bin/time where applicable
Project acquisition	Git, BugsCpp, Docker
Output formats	Terminal report, text report, CSV

The framework relied mainly on standard C++ features and external command-line tools. Source-code discovery used supported C and C++ file extensions such as .c, .cpp, .h, and .hpp. Non-code files such as documentation, metadata, images, and version-control files were not treated as analysis targets by the framework.

Docker was used during dataset acquisition because BugsCpp uses Docker-based project environments. Git was used to clone and manage the underlying repositories. The actual PatchImpact analysis was performed on the checked-out buggy and fixed folders produced by BugsCpp.

The experimental environment is an important part of the reproducibility of this thesis. Static analysis results, Cppcheck findings, runtime measurements, and executable behaviour may depend on tool versions, compiler versions, operating system behaviour, and available system resources. For this reason, the evaluation should be interpreted as a controlled experiment within the stated Linux environment rather than as a universal measurement independent of platform.

5.3 Measurement Procedure

The measurement procedure followed a repeatable command-line workflow. The first stage was dataset acquisition. BugsCpp was used to check out buggy and fixed versions of selected open-source C and C++ projects. Each defect instance produced a pair of directories, usually named buggy-N and fixed-N, where N identifies the defect instance.

The second stage was validation of available pairs. Each project folder was inspected to determine how many buggy and fixed versions had been successfully downloaded. Only pairs where both the buggy and fixed version were available were considered complete patch pairs. This step was necessary because dataset acquisition could sometimes produce partial results when a checkout, build image, or dependency failed.

The third stage was folder comparison. In this mode, PatchImpact was executed with two folder paths: the buggy version as the original input and the fixed version as the patched input. A typical command had the following form:

./patchimpact_v41 -rep ~/PatchImpactResults/berry_1_report.txt ~/ThesisDatasets/bugscpp/berry/buggy-1 ~/ThesisDatasets/bugscpp/berry/fixed-1

This command produced a human-readable report containing file-level comparisons, project-level summary values, structural impact, quality impact, and the final overall patch evaluation. Report generation was used because it preserved detailed evidence about each comparison and allowed the final score, grade, and risk level to be verified manually.

The fourth stage was mass assessment. In this mode, PatchImpact was executed on a prepared folder containing compatible artefacts. The command had the following form:

./patchimpact_v41 -csv mini_mass.csv -ma ~/PatchImpactMini

This produced a structured CSV file containing pairwise directional comparisons. For source pairs, the CSV included values such as complexity change, function change, SLOC change, similarity percentage, maintainability change, Cppcheck change, structural score, and quality score. For executable pairs, the CSV included runtime and binary-related fields such as elapsed time, CPU time, memory usage, binary size, exit status, and dynamic score.

The fifth stage was aggregation. The CSV output was processed to count the number of generated rows and to calculate average structural and quality scores. The mass assessment CSV contained 28,436 data rows. Of these, 28,056 were source-pair comparisons and 380 were executable-pair comparisons. The average structural score across the numeric source-pair rows was 94.46 out of 100, while the average quality score was 95.75 out of 100.

The measurement procedure therefore combined detailed per-pair reports with larger CSV-based aggregation. The report files were used to verify final patch-

level grades and risk levels, while the CSV output was used for broader quantitative summaries. This combination was useful because the report format provided interpretability, while the CSV format supported tabular analysis and later inclusion in the thesis appendices.

Table 5.3 Measurement procedure

Step	Description
1	Download buggy and fixed versions using BugsCpp
2	Verify complete pairs where both buggy and fixed folders exist
3	Run PatchImpact folder comparison on selected bug-fix pairs
4	Generate text reports using -rep
5	Run mass assessment using -ma on a prepared artefact folder
6	Export mass assessment data to CSV
7	Aggregate row counts and score summaries
8	Use reports and CSV output as evidence for results and discussion

This workflow allowed PatchImpact to be evaluated both as a direct patch-comparison tool and as a larger-scale metric generation framework. The folder comparison experiments show how the framework behaves on real bug-fix pairs, while the mass assessment experiment shows that it can produce larger structured datasets for exploratory analysis.

Table 5.4 summarises the preliminary mass assessment results used as evidence that the framework can generate and export large quantities of structured metric data.

Table 5.4 Mini mass assessment summary

Metric	Value
Total comparison rows	28,436
Source-pair rows	28,056
Executable-pair rows	380
Average structural score	94.46 / 100
Average quality score	95.75 / 100

The results in Table 5.4 should be interpreted as exploratory mass-assessment output rather than as a count of independent bug-fix patches. They show that

the framework can process a large number of pairwise artefact comparisons and produce structured output for later analysis. The final patch-level interpretation remains based on the paired buggy and fixed project reports.

Results

This chapter presents the results produced by the PatchImpact framework during the experimental evaluation. The results are reported in two main groups. The first group consists of direct folder comparison results for complete buggy and fixed project pairs. The second group consists of the mini mass assessment results generated from a prepared folder of source and executable artefacts. Together, these results show how the framework behaves both in direct patch-pair comparison and in larger pairwise artefact assessment.

The purpose of this chapter is to report the measured outcomes. Detailed interpretation, limitations, and broader implications are discussed in the following analysis and discussion chapter. For this reason, this chapter focuses mainly on the observed values, generated scores, and measurable outputs of the framework.

A distinction is maintained between complete final patch evaluations and exploratory mass assessment rows. The complete patch evaluations are based on report files that reached the final overall patch evaluation section. The mass assessment CSV, in contrast, contains many pairwise directional comparisons between compatible artefacts and is used mainly for aggregate metric analysis. Therefore, mass assessment rows should not all be interpreted as independent real world patches through some are.

6.1 Overview of Available Results

The available results consisted of five complete Berry patch reports, one incomplete Coreutils report, a summary CSV file, and one mini mass assessment CSV file. The Berry reports contained full project-level comparison summaries, structural and quality scores, overall patch scores, grades, and risk levels. These reports are therefore used as the main completed patch-pair results in this chapter.

The Coreutils report was generated during the experiment but did not reach the final overall evaluation section. It stopped while still listing file-level comparisons and did not contain an overall patch score, final grade, or risk level. For this reason, it is not included in the final scored patch comparison table. However, it remains useful as practical evidence that very large projects can produce expensive and long-running analysis outputs.

Result artefact	**Status**	**Use in this chapter**
Berry reports 1 to 5	Complete	Used for scored folder comparison results
Coreutils 1 report	Incomplete final summary	Excluded from final scoring, mentioned as scalability observation

Result artefact	Status	Use in this chapter
summary.csv	Complete for Berry reports	Used to verify Berry overall scores, grades, and risks
mini_mass.csv	Complete	Used for aggregate pairwise metric analysis
mini_mass_summary.txt	Complete	Used to report aggregate row counts and average scores

6.2 Folder Comparison Results

The direct folder comparison experiment evaluated five Berry bug-fix pairs. Each pair compared a buggy version against its corresponding fixed version. The framework scanned the supported C and C++ artefacts, matched files by relative path, detected changed and unchanged files, calculated static metric differences, and generated an overall patch evaluation.

The complete Berry patch results are shown in Table 6.1. All five patches received Grade A and Low risk. The overall scores were very high, ranging from 97.4992 to 100.0000. This indicates that, according to the implemented heuristic model, the measured changes in these patch pairs were small and controlled at project level.

Patch pair	Matched files	Changed files	Unchanged files	Overall score	Grade	Risk
Berry-1	84	1	83	100.0000	A	Low
Berry-2	84	1	83	99.9993	A	Low
Berry-3	84	1	83	97.4992	A	Low
Berry-4	83	1	82	99.9895	A	Low
Berry-5	82	1	81	99.9922	A	Low

Table 6.1 shows that each Berry patch changed only one matched file while the majority of files remained unchanged. This is important because the folder comparison mode evaluates the project-level footprint of the patch rather than only the edited file. In these five cases, the measured patch impact was localised, and the overall project-level structure was mostly preserved.

6.3 Structural Impact Results

The structural impact score was calculated from source-level differences such as complexity change, function change, source-line change, and similarity. The Berry results showed very small structural differences. Berry-1 had no measured complexity, function, or SLOC change and therefore received a structural score of 100.0000. The other Berry patches also received structural scores very close to 100.

Patch pair	Added lines	Removed lines	Complexity change	Function change	SLOC change	Structural score
Berry-1	1	1	0	0	0	100.0000
Berry-2	1	0	0	0	1	99.9985
Berry-3	2	1	0	0	1	99.9984
Berry-4	5	1	1	0	4	99.9790
Berry-5	1	1	1	0	0	99.9845

The lowest structural score among the Berry cases was 99.9790 for Berry-4. This was still very high, but it reflected a larger structural difference than the other cases because the patch added five lines, removed one line, increased complexity by one, and increased SLOC by four. Berry-5 also showed a complexity increase of one, but without a SLOC increase. These results demonstrate that the framework can distinguish between patches that are all low risk but still differ slightly in structural footprint.

Across the five complete Berry reports, the average structural score was approximately 99.9921 out of 100. This high average reflects the fact that the analysed Berry patches were small and localised relative to the whole project.

6.4 Quality Impact Results

The quality impact score was influenced by static quality indicators such as Cppcheck finding changes and build-related information where available. Four of the five Berry patches produced a quality score of 100.0000. Berry-3 produced a lower quality score of 95.0000 because the matched Cppcheck total change increased by one.

Patch pair	Cppcheck total change	Quality score	Grade	Risk
Berry-1	0	100.0000	A	Low
Berry-2	0	100.0000	A	Low
Berry-3	1	95.0000	A	Low
Berry-4	0	100.0000	A	Low
Berry-5	0	100.0000	A	Low

Berry-3 is the most interesting Berry result because its structural score remained almost perfect, but its quality score was lower than the other four cases. This caused its overall patch score to drop to 97.4992, making it the lowest overall score among the complete Berry reports. This result shows that the heuristic model does not rely only on size or complexity changes. Static quality signals can also influence the final evaluation.

The average quality score across the five complete Berry reports was 99.0000 out of 100. The average overall score across the five complete Berry reports was approximately 99.4960 out of 100.

6.5 Mini Mass Assessment Results

In addition to direct folder comparison, the framework was evaluated using mass assessment mode on a prepared folder of source and executable artefacts. This mode performs pairwise directional comparisons between compatible artefacts discovered in the selected folder. It is useful for exploratory analysis because it can generate a much larger number of metric observations than a small number of direct patch reports.

The mini mass assessment generated 28,436 data rows. Of these, 28,056 were source-pair comparisons and 380 were executable-pair comparisons. The source-pair rows contained structural and quality fields such as complexity change, function change, SLOC change, similarity percentage, maintainability change, Cppcheck change, structural score, and quality score. The executable-pair rows contained dynamic or binary-related fields such as elapsed time, CPU time, memory usage, binary size, exit status, and dynamic score where applicable.

Metric	**Value**
Total mass assessment rows	28,436
Source-pair rows	28,056
Executable-pair rows	380
Average structural score	94.46 / 100
Average quality score	95.75 / 100

The average structural score across the numeric source-pair rows was 94.46 out of 100. The average quality score was 95.75 out of 100. These values are lower than the Berry folder comparison averages because the mass assessment mode compares many possible artefact pairs rather than only carefully matched bug-fix versions. This is expected, since arbitrary pairwise comparisons can include artefacts that differ more substantially from one another than a normal patch pair.

The result demonstrates that the framework was able to generate a large structured CSV file and process tens of thousands of pairwise source-code comparisons. It also shows that the framework can produce both detailed report files and aggregate tabular data suitable for later analysis.

6.6 Dynamic and Executable Results

Dynamic and executable-level assessment was represented in the mini mass assessment by 380 executable-pair rows. These rows show that the framework was capable of discovering executable artefacts and including them in the mass assessment output. The dynamic fields are separate from the source-pair structural and quality fields because runtime behaviour can only be measured meaningfully for executable artefacts.

In the direct Berry folder comparison reports, the dynamic or behavioural score was reported as not available because folder comparison mode did not execute paired project binaries as part of the final patch score. In this implementation version, dynamic measurements are supported through executable-level assessment and mass assessment, while folder comparison primarily reports structural and quality impact unless explicit executable comparisons are performed.

This distinction is important for interpreting the results. The thesis framework supports both static and dynamic analysis, but not every experiment produces every metric. Source folder comparisons provide project-level static and quality evidence. Executable comparisons provide runtime and binary evidence. The results should therefore be read according to the type of artefact and analysis mode used.

6.7 Summary of Main Results

The complete Berry folder comparison results show that the framework successfully generated patch-level reports with structural scores, quality scores, overall scores, grades, and risk levels. All five complete Berry patches received Grade A and Low risk. Their average overall score was approximately 99.4960 out of 100, their average structural score was approximately 99.9921 out of 100, and their average quality score was 99.0000 out of 100.

The mini mass assessment results show that PatchImpact can also generate large CSV-based outputs for exploratory analysis. The experiment produced 28,436 rows, including 28,056 source-pair comparisons and 380 executable-pair comparisons. The aggregate source-pair results produced an average structural score of 94.46 out of 100 and an average quality score of 95.75 out of 100.

Result category	**Main finding**
Complete Berry patch reports	Five complete scored patch comparisons
Berry grades and risk levels	All Grade A and Low risk

Result category	**Main finding**
Average Berry overall score	Approximately 99.4960 / 100
Average Berry structural score	Approximately 99.9921 / 100
Average Berry quality score	99.0000 / 100
Mini mass assessment size	28,436 pairwise rows
Mini mass source-pair rows	28,056 rows
Mini mass executable-pair rows	380 rows
Mini mass average structural score	94.46 / 100
Mini mass average quality score	95.75 / 100

Overall, the results show that the framework can be used in two complementary ways. First, it can evaluate complete buggy and fixed project pairs and produce interpretable patch-level scores. Second, it can generate larger CSV datasets from pairwise mass assessment for exploratory metric analysis. These results provide the empirical basis for the discussion in the next chapter, where the meaning, usefulness, and limitations of the framework are examined in relation to the thesis research questions.

Analysis and Discussion

This chapter interprets the experimental results presented in Chapter 6 and explains what they mean for the study of software patch impact. The previous chapter reported the measured outputs of the PatchImpact framework, including complete Berry patch reports and a mini mass assessment. This chapter moves from reporting to interpretation. It discusses what the scores suggest about patch effects, how these findings relate to the software engineering literature, and what knowledge the thesis contributes beyond the implementation of a working tool.

The main argument of this chapter is that a software patch should not be understood only as a textual difference or a pass or fail outcome. A patch can also be interpreted as a measurable disturbance to a software system. This disturbance may appear in the structure of the source code, in static quality indicators, in build behaviour, or in runtime behaviour when executable analysis is available. This interpretation connects the experimental results to the broader literature on software metrics, patch explanation, patch overfitting, code review, and empirical software engineering.

7.1 From Tool Output to Patch Impact Knowledge

The results of the framework are useful only if they are interpreted as evidence about patch impact rather than as isolated numbers. A score of 100 or 95 does not, by itself, explain whether a patch is correct, maintainable, or desirable. Its meaning comes from the relationship between the original version, the patched version, the metrics that changed, and the expected role of the patch. This is why the discussion must connect the framework output to the concept of patch effect.

The contribution of the thesis is therefore not simply that a program produced reports and CSV files. The contribution is the construction and evaluation of an interpretable method for turning source-level and executable-level measurements into patch-level evidence. The framework makes it possible to ask not only whether two versions differ, but how they differ, how much they differ, and which measured dimensions were affected.

This matters because previous work has shown that patch evaluation cannot rely only on surface success. Smith et al. (2015) discuss the problem of overfitting in automated program repair, where a patch may satisfy available tests but still fail to represent a correct general repair. Martínez et al. (2017) also show that large-scale automatic repair experiments require careful interpretation of patch correctness and quality. These observations support the need for evaluation methods that provide more evidence than pass or fail validation alone.

7.2 Patch Effect as Measurable Distance

One of the main conceptual outcomes of the thesis is the interpretation of patch impact as a measurable distance between program versions. In this context, distance does not mean physical distance or only textual difference. It means the degree to which the patched version moves away from the original version in terms of measured structure, quality, and behaviour.

This interpretation is grounded in the use of software metrics. McCabe (1976) introduced cyclomatic complexity as a quantitative measure based on control flow and connected it to testing and maintainability concerns. This is directly relevant to patch evaluation because a patch that changes decision structure can increase the number of possible paths and therefore affect understandability, testability, or maintenance effort. However, complexity alone is not enough. Trautsch et al. (2023) show that static metric values and static analysis warnings can provide evidence about quality-related changes, but their relationship with quality must be interpreted carefully. This supports the decision to combine multiple metrics rather than relying on a single indicator.

In the framework, this measurable distance is expressed through changes in complexity, function count, SLOC, similarity, maintainability-related values, Cppcheck findings, build status, and executable behaviour where available. The overall score is therefore best understood as an interpretation of multiple partial distances. It is not a mathematical proof of correctness, but a structured signal about how disruptive the patch appears according to the selected evidence.

Dimension of patch effect	**Examples of measured indicators**	**Interpretation**
Structural effect	Complexity change, function change, SLOC change, similarity	Shows how much the source-code structure changed.
Quality effect	Cppcheck findings, maintainability-related changes, build status	Shows whether the patch introduced or removed static quality concerns.
Behavioural effect	Execution time, CPU time, memory usage, binary size, exit status	Shows runtime or executable-level change when executable analysis is available.
Overall heuristic effect	Structural score, quality score, dynamic score where measured, grade, risk level	Summarises the measured disturbance into an interpretable decision-support signal.

7.3 Interpretation of the Berry Patch Results

The complete Berry results showed five complete patch-pair evaluations. All five received Grade A and Low risk. Their average overall score was approximately 99.4960 out of 100, their average structural score was approximately 99.9921 out of 100, and their average quality score was 99.0000 out of 100. These results indicate that the measured effects of the Berry patches were small relative to the surrounding project structure.

This finding should not be interpreted as meaning that the Berry patches were unimportant. A patch can be important because it fixes a real defect while still being small in measured impact. In fact, this can be a desirable property. A focused bug-fix patch should ideally correct the defect without unnecessarily disturbing unrelated parts of the system. The Berry results therefore support the idea that low impact can be a sign of localisation and control rather than insignificance.

This interpretation also connects to the distinction between corrective and quality-improving change. Trautsch et al. (2023) report that different maintenance intentions can affect static metrics differently and that corrective changes may add complexity. The Berry cases analysed in this thesis show a pattern of very limited measured disturbance. This does not contradict the literature. Instead, it demonstrates why measuring each patch is useful: some bug fixes may have broad structural footprints, while others may remain highly localised.

Patch pair	**Overall score**	**Structural score**	**Quality score**	**Grade**	**Risk**
Berry-1	100.0000	100.0000	100.0000	A	Low
Berry-2	99.9993	99.9985	100.0000	A	Low
Berry-3	97.4992	99.9984	95.0000	A	Low
Berry-4	99.9895	99.9790	100.0000	A	Low
Berry-5	99.9922	99.9845	100.0000	A	Low

7.4 Structural Impact and Quality Impact Are Different Signals

A particularly important observation from the Berry results is that structural impact and quality impact are not the same thing. Berry-3 is the clearest example. Its structural score remained almost perfect, but its quality score decreased to 95.0000 because the static quality indicators changed. As a result, Berry-3 received the lowest overall score among the complete Berry patch pairs, even though its structural score remained high.

This result is important because it shows that a patch can be structurally small while still affecting quality-related indicators. If the evaluation had considered only SLOC, complexity, or similarity, Berry-3 would have appeared almost iden-

tical to the other Berry cases. By including static quality evidence, the framework identified a difference that would otherwise have been hidden.

This supports a central claim of the thesis: patch impact is multi-dimensional. Complexity, size, similarity, function count, warnings, and runtime behaviour each describe a different aspect of change. A single metric may be useful, but it is rarely sufficient. This is consistent with Trautsch et al. (2023), who show that static metrics and static analysis warnings can provide useful but incomplete evidence about code quality change. It also connects to code review research, since maintainability and reviewability are often part of the evaluation of a change, not only functional correctness.

7.5 Meaning of the Mini Mass Assessment Results

The mini mass assessment produced 28,436 comparison rows. These included 28,056 source-pair rows and 380 executable-pair rows. The average structural score across the numeric source-pair rows was 94.46 out of 100, and the average quality score was 95.75 out of 100.

These results must be interpreted carefully. The mass assessment rows do not represent 28,436 independent real-world patches. They represent directional pairwise comparisons between compatible artefacts discovered inside the prepared experiment folder. This makes mass assessment useful for exploratory analysis, but different from direct buggy-versus-fixed patch evaluation.

The lower average structural and quality scores in the mass assessment are expected because arbitrary artefact comparisons may differ more substantially than actual patch pairs. The value of this result is therefore methodological. It shows that the framework can generate large structured outputs, separate source-pair and executable-pair rows, and provide data suitable for aggregation. This supports the use of PatchImpact as an empirical software engineering instrument rather than only as a single-case demonstration.

Mass assessment metric	**Value**
Total comparison rows	28,436
Source-pair rows	28,056
Executable-pair rows	380
Average structural score	94.46 / 100
Average quality score	95.75 / 100

7.6 Dynamic and Behavioural Interpretation

The thesis aimed to combine static and dynamic perspectives. The experimental results show that this goal was achieved partly. Executable-level assessment was represented in the mini mass assessment by 380 executable-pair rows. However, the direct Berry folder comparison reports did not include behavioural scores because the folder comparison mode did not execute paired project binaries as part of the final folder-level patch score.

This distinction is important. The framework supports dynamic measurement, but not every analysis mode produces dynamic evidence. Folder comparison is useful because it can be applied broadly to source trees, even when compiling the full project is difficult. Dynamic analysis is more demanding because it depends on successful builds, executable artefacts, meaningful inputs, runtime conditions, and workload design.

This limitation is consistent with the general distinction between static and dynamic analysis. Static analysis can provide broad source-level evidence without executing the program, while dynamic analysis provides operational evidence only under particular execution conditions. For patch evaluation, the two forms of evidence are complementary. Static measurements can show structural and quality disturbance, while dynamic measurements can reveal runtime disturbance when executable artefacts are available.

7.7 Relation to Patch Explanation and Review

Another important contribution of the framework is that it produces explanation-oriented results. A final score is useful only if the user can understand why the score was produced. The report files generated by the framework show the measured components behind the score, including structural change, quality change, and whether behavioural evidence was available. This makes the output more useful for human review than a single unexplained label.

This connects directly to patch explanation research. Liang et al. (2019) argue that developers need explanations to understand patches and that automated repair approaches should not only produce patches, but also support patch explanation. Their empirical study identifies elements that commonly appear in patch explanations, such as condition, consequence, position, cause, and change. PatchImpact does not generate natural language explanations of bug causes, but it contributes a metric-based explanation of change. It tells the reviewer what changed according to the selected measurements and where the measurable disturbance appears.

This is also important because patch evaluation remains connected to human judgement. A low-risk score should not mean that a developer can ignore review. Instead, it can help prioritise attention. A patch with low measured disturbance may still need normal review and testing, while a patch with increased warnings, build changes, or large structural distance may deserve closer inspection.

7.8 Answering the Research Questions

The results provide evidence for answering the four research questions of the thesis. The answers should be interpreted cautiously because the complete scored patch set was limited, but they still show how the framework contributes to the study of patch impact.

RQ1. How can static and dynamic metrics quantify structural and behavioural effects of software patches?

The results show that static metrics can quantify structural effects such as complexity change, SLOC change, function change, similarity, and static quality findings. Dynamic metrics can quantify behavioural effects when executable artefacts are available, including runtime and binary-related measurements. In the current experiments, folder comparison mode mainly quantified structural and quality effects, while executable-related measurements appeared in the mass assessment output. Therefore, static and dynamic metrics can quantify different dimensions of patch effect, but their availability depends on the analysis mode and artefacts used.

RQ2. Can heuristic-based models distinguish high-quality patches from poor or overfitted patches?

The results provide partial support for this question. The heuristic model distinguished Berry-3 from the other Berry patches because its quality score decreased, even though its structural score remained high. This shows that the model can detect differences that are not visible from structural size alone. However, the experiment does not prove full reliability for detecting poorly designed or overfitted patches in all cases. A metric-based heuristic can support ranking and triage, but it cannot by itself prove semantic correctness or detect every form of overfitting.

RQ3. Which metrics are most effective for C and C++ patch impact assessment?

The most practical metrics were source-level structural and quality indicators because they can be collected without building the whole project. Complexity, SLOC, similarity, function count, and Cppcheck changes were useful for distinguishing patch footprint and quality effects. Runtime metrics such as execution time, memory usage, binary size, and exit status are valuable, but they require executable artefacts and controlled execution conditions. Therefore, the strongest approach is not a single metric, but a layered combination of source-level metrics, static analysis findings, build information, and executable-level measurements where available.

RQ4. How can a metric-based framework assist developers?

The framework can assist developers by providing structured evidence about the consequences of a patch. It can show whether a patch changed complexity, added or removed code, changed similarity, introduced static analysis findings, affected build status, or changed executable behaviour where measured. This can help developers decide whether a patch appears focused, whether it requires closer review, and whether its measured impact is proportionate to the intended fix. The framework therefore acts as a triage and explanation tool rather than as a replacement for testing or code review.

7.9 Limitations and Threats to Validity

Several limitations must be acknowledged. The mass assessment results should not be treated as thousands of independent real-world patches. They are directional pairwise artefact comparisons. They are useful for exploratory metric aggregation and scalability observation, but they are different from direct bug-fix pair evaluation.

The framework depends on external tools and the execution environment. Static analysis findings may vary with Cppcheck versions, complexity values may depend on parsing behaviour, and runtime measurements can depend on the operating system, hardware, workload, and input selection. Fifth, the heuristic model depends on selected thresholds and weights. These choices make the model interpretable, but they also mean that the score is a decision support signal rather than an objective universal truth.

Finally, software metrics cannot fully capture semantic correctness. A patch can receive a high score and still be logically wrong if it does not actually fix the intended defect. This limitation is consistent with research on patch overfitting and weak test validation. Metrics can support review, but they cannot replace semantic reasoning, testing, or human understanding.

7.10 Summary of the Chapter

This chapter interpreted the results as evidence about patch effect. The Berry results suggest that real bug fix patches can be important while still having low measured structural disturbance. The Berry 3 case shows that structural and quality effects should be measured separately because a patch can remain structurally small while still changing static quality indicators. The mini mass assessment shows that the framework can generate larger structured datasets for exploratory analysis, but only correctly paired artifacts give the most value still this happens as renaming is a concern and some times 2 distinct codes can be closer in architecture than the code and its patch .

The main knowledge contribution is that patch impact can be treated as a measurable and interpretable combination of structural, quality, and behavioural change. The framework does not prove patch correctness. Its value is that it makes patch effects visible, comparable, and explainable. This supports devel-

opers and researchers in reasoning about patch quality beyond textual diff size, isolated metrics, manual judgement, or test suite success alone.

Conclusion

This thesis investigated how the impact of software patches can be evaluated in a systematic, measurable, and reproducible way. The work began from a central problem in software engineering practice: software systems evolve through patches, but the consequences of those patches are not always visible through compilation, testing, or manual inspection alone. A patch may repair a defect and still change control flow, increase complexity, introduce new static analysis findings, affect runtime behaviour, or create future maintenance risk. For this reason, patch evaluation should not be limited to asking whether the code changed or whether the patched version appears to work. It should also ask how strongly the patch affects the structure, quality, and behaviour of the software system.

The thesis addressed this problem by designing, implementing, and evaluating PatchImpact, a tool-supported framework for measuring software patch impact in open-source C and C++ projects. PatchImpact compares original and patched artefacts, extracts measurable software metrics, applies heuristic scoring, and produces both human-readable reports and structured spreadsheet-compatible outputs. This means that the thesis does not only present a theoretical argument about patch evaluation. It also delivers a functioning experimental framework that can be executed, inspected, repeated, and extended.

The work therefore fulfils the central promise of the project. The original aim was to combine static and dynamic measurements, estimate logical distance between software versions, evaluate whether patches remain minimally invasive, and support patch comparison through an interpretable grading model. The completed framework implements these ideas as a practical research instrument. It supports folder comparison, individual assessment, mass assessment, report generation, structured export, build-related analysis, and executable-level dynamic measurement. This makes PatchImpact more than a small utility. It is an integrated framework that connects software measurement, patch comparison, and heuristic interpretation.

The conceptual contribution of the thesis is the interpretation of a patch as a measurable disturbance to a software system. A low-impact patch is not necessarily unimportant. It may represent a focused repair that corrects a defect while preserving the wider project structure. A high-impact patch is not automatically bad either, because some important fixes require substantial change. However, a higher measured impact indicates that the patch changes more of the system or affects more quality indicators, and therefore deserves closer inspection. This interpretation gives developers and researchers a more precise way to reason about patches than simple textual difference, pass-or-fail validation, or informal judgement.

This interpretation is consistent with established software engineering research. McCabe's work on cyclomatic complexity showed that program structure and control flow can be measured quantitatively and related to testing and main-

tainability concerns (McCabe, 1976). Later empirical work has also shown that static metrics and static analysis warnings can provide useful signals about quality evolution, while also requiring careful interpretation rather than blind acceptance (Trautsch, Erbel, Herbold, & Grabowski, 2023). PatchImpact follows this line of reasoning by treating metrics as evidence for interpretation, not as absolute proof of correctness.

The empirical results support the usefulness of this approach. The complete Berry patch comparisons produced consistently strong results, with all five complete cases receiving the highest grade and low risk. The important point is not merely that the scores were high. The important point is what the scores reveal: the analysed patches were measured as localised and low-disruption changes at project level. This supports the view that a well-designed bug fix may be structurally conservative while still being meaningful. In practical terms, a good patch does not need to be large or invasive. It should make the necessary correction while limiting unnecessary disturbance to the surrounding system.

The Berry-3 case is especially important because it shows why patch impact must be treated as multi-dimensional. Berry-3 retained a very high structural score but received a lower quality score because of a change in static analysis findings. This demonstrates that structural impact and quality impact are related but not identical. A patch can preserve most of the surrounding structure while still affecting static quality indicators. If the analysis had considered only complexity, source lines, or similarity, this difference would have been less visible. By combining structural and quality evidence, the framework provides a more informative assessment of the patch effect.

The mini mass assessment further demonstrated the practical strength of the framework. The produced output contained 28,436 rows, including 28,056 source-pair comparisons and 380 executable-pair comparisons. These rows should not be interpreted as thousands of independent real-world patches. They represent pairwise directional comparisons between compatible artefacts inside the prepared dataset. Nevertheless, the result is important because it shows that PatchImpact can generate larger structured datasets for exploratory analysis. Empirical software engineering depends not only on isolated examples, but also on repeatable workflows for producing and examining evidence. PatchImpact provides such a workflow.

The thesis also shows that static and dynamic analysis are complementary rather than interchangeable. Static analysis is broadly applicable to source folders and can reveal structural and quality-related effects even when building a full project is difficult. Dynamic analysis provides behavioural evidence when executable artefacts are available, including information about runtime cost, memory use, binary size, and exit status. The experiments show that dynamic evidence is valuable, but also more dependent on build success, executable availability, input selection, and controlled runtime conditions. This is an honest and important finding, not a weakness of the thesis, because it reflects a real challenge in evaluating C and C++ software systems.

The work therefore makes a practical contribution to patch evaluation. PatchImpact can assist developers by making patch effects visible and explainable. It can show whether complexity changed, whether functions or source lines were added or removed, whether similarity decreased, whether static analysis findings changed, whether build status was affected, and whether executable behaviour changed where such measurement is possible. This gives developers a stronger basis for review and triage. The framework does not replace testing, code review, or expert judgement. Instead, it strengthens them by adding a measurable layer of evidence.

This matters because existing patch evaluation practices have known limitations. Testing is essential, but a patch that passes available tests may still be incomplete, overfitted, or semantically weak (Martínez, Durieux, Sommerard, Xuan, & Monperrus, 2016; Smith, Barr, Goues, & Brun, 2015). Manual review is also important, but it is time-consuming and depends on human judgement and project knowledge (Ye, Martínez, & Monperrus, 2021). Patch explanation research further shows that developers need to understand why a patch is acceptable, not merely receive a changed code fragment (Liang et al., 2019). PatchImpact contributes to this need by producing reports that connect final scores to the measurements behind them.

The thesis should also be understood as an engineering achievement. The framework was implemented as a modular command-line tool in C++17, integrated with external analysis tools, applied to real open-source patch data, and used to generate concrete results. The work required not only literature review and conceptual modelling, but also practical implementation, dataset preparation, command-line experimentation, result extraction, and critical interpretation. This combination of theory, tool construction, and empirical evaluation gives the thesis value as a software engineering project, not only as a written discussion.

There are, of course, limitations. The complete scored folder-comparison results were limited to the successful Berry cases, while the Coreutils report did not reach a final evaluation section and was therefore excluded from final scored interpretation. Dynamic behavioural evidence was supported by the framework and appeared in executable-level assessment, but it was not fully integrated into every folder-comparison score. The mass assessment output represents pairwise artefact comparisons rather than independent patches. The framework also depends on external tools and on the experimental environment, meaning that results may vary with different tool versions, compiler versions, operating systems, and runtime conditions.

These limitations do not undermine the contribution. They define its realistic scope. Evaluating real C and C++ patches is difficult because projects vary in size, build systems, dependencies, input requirements, and executable behaviour. The thesis confronts this difficulty directly and provides a framework that measures what can be measured in a reproducible and extensible way. It also identifies where stronger future work is needed, especially in fuller dynamic

execution, broader datasets, improved build integration, and deeper semantic validation.

The main contribution of this thesis is therefore a reproducible and interpretable approach for evaluating software patch impact as a combination of structural, quality, and behavioural change. PatchImpact shows that patches can be analysed not only as code edits, but as measurable interventions in a software system. It gives developers and researchers a way to reason about whether a patch is focused, whether it introduces unnecessary disturbance, and whether it deserves closer inspection.

Ultimately, the value of this work is that it makes patch impact more visible. It moves patch evaluation away from purely informal judgement and toward measurable, explainable, and repeatable analysis. It does not claim to solve patch correctness completely, and it does not claim to replace existing software engineering practices. Its contribution is more practical and more defensible: it adds an evidence-based layer that supports testing, review, and expert reasoning. By doing so, the thesis provides both a working framework and a clear conceptual model for understanding how software patches affect C and C++ projects.

Future Work

1. Expanding to More Programming Languages

The current framework focuses on C and C++ because they are important for systems programming, performance-sensitive software, and low-level resource management. Future work could extend the framework to additional languages such as Rust, Java, C#, Go, and Python.

This would make the framework more general and allow comparison between different programming ecosystems. For example, Rust would be interesting because of its memory-safety model, while Java and C# would allow analysis of object-oriented systems with different runtime environments. Supporting more languages would require language-specific parsers, metric tools, build systems, and execution models, but the general idea of comparing original and patched versions through measurable indicators would remain useful.

2. Improving Support for Larger Datasets

A major future direction is the use of larger and more diverse patch datasets. This thesis used real and controlled patch examples to evaluate the framework, but future work could apply the framework to many more projects and patch pairs from larger repositories.

A larger dataset would make it possible to study patterns across different types of patches, projects, and domains. It would also allow stronger empirical conclusions about which metrics are most useful for identifying low-impact, high-impact, risky, or noisy patches. Future work could also improve automation for dataset preparation, including automatic detection of original and patched pairs, filtering of incomplete projects, and better handling of projects that are difficult to build.

3. Better Handling of Very Large Projects

The current work showed that large C and C++ projects can be difficult to analyse fully because they may contain many files, external dependencies, generated code, build scripts, and platform-specific configurations. Future versions of the framework could include better strategies for large-scale project analysis.

This could include analysing only files affected by the patch, separating project code from third-party code, using timeouts, caching previous metric results, and allowing users to define analysis scopes. Another improvement would be incremental analysis, where the framework avoids re-analysing unchanged files and focuses more directly on the patch-relevant parts of the project. This would make the framework faster and more practical for industrial-scale repositories.

4. More Precise Patch Targeting

Future work could improve the framework's ability to identify exactly which parts of a project are affected by a patch. The current approach compares matching source files and project folders, which is useful for measuring project-level impact. However, a more advanced version could combine folder comparison with patch-diff analysis, commit metadata, function-level mapping, and dependency tracking.

This would allow the framework to distinguish between direct changes and indirect effects. For example, a patch may modify one function but affect other functions that depend on it. More precise targeting would help the framework explain whether a patch is local, function-level, file-level, module-level, or project-wide. This would strengthen the idea of logical distance between software versions.

5. Expanding the Metric Set

Another future direction is to expand the set of metrics used by the framework. The current version focuses on structural, quality, and executable-level measurements such as complexity, source lines of code, static analysis findings, execution time, memory usage, binary size, and exit status. Future work could add new categories of metrics.

Possible additions include coupling, cohesion, dependency metrics, code churn, test coverage, mutation score, security warnings, compiler warnings, build time, energy consumption, and runtime profiling data. For object-oriented languages, future work could also include inheritance depth, class coupling, method complexity, and design-smell indicators. These additions would allow the framework to evaluate patch impact from more perspectives.

6. Stronger Dynamic and Behavioural Analysis

Future work could improve the dynamic analysis part of the framework. Dynamic measurements are valuable because some patch effects only appear when the program runs. However, runtime analysis depends on available executables, suitable inputs, repeatable workloads, and stable measurement conditions.

A future version could include automatic workload generation, repeated benchmark execution, statistical summaries, input configuration files, and more detailed profiling. It could also compare program outputs, exit behaviour, logs, error messages, and resource usage patterns. This would make behavioural impact analysis stronger and would help connect metric changes to actual runtime consequences.

7. Better Heuristic Calibration

The heuristic scoring model could also be improved through calibration on larger datasets. In this thesis, the model was designed to be interpretable and practical, but future work could refine the thresholds and weights using more empirical evidence.

For example, future research could compare framework scores against expert developer judgement, code review outcomes, reverted patches, failed builds, or known defective patches. This would help determine whether certain metric changes should be penalised more strongly than others. The model could also support different scoring profiles depending on the project context, such as safety-critical systems, performance-sensitive software, or general application code.

8. More Advanced Risk Classification

Future work could expand the grading and risk interpretation system. The current grading model provides an understandable summary of patch impact, but more detailed classifications could be added.

For example, the framework could classify patches as low structural risk, high quality risk, high behavioural risk, performance regression risk, maintainability risk, or suspicious deletion risk. This would give developers more useful explanations than a single overall grade. A patch that removes many lines of code, for example, may look simpler but could also remove important functionality. More detailed risk categories would help avoid misleading conclusions.

9. Integration with Development Tools

A practical future direction is integration with development workflows. PatchImpact is currently a command-line research tool, which is useful for reproducible experiments. Future work could integrate it with continuous integration pipelines, code review systems, Git hooks, repository dashboards, or integrated development environments.

This would allow developers to run patch impact analysis automatically when a pull request is created or when a patch is submitted. The framework could then provide a report showing whether the patch is small and focused, whether it introduces new warnings, whether it changes runtime behaviour, or whether it requires closer review. This would make the framework useful not only as a research prototype but also as a practical software engineering support tool.

10. Improved Visualisation and Reporting

Future versions of the framework could also improve the way results are presented. The current reports and CSV outputs are useful for analysis, but visual dashboards could make the results easier to understand.

Possible visualisations include patch impact charts, metric-difference graphs, grade distributions, risk heatmaps, file-level impact maps, and before-and-after comparison summaries. This would help developers and researchers quickly identify where the main changes occurred and which patches deserve closer attention.

11. More Controlled Synthetic Patch Scenarios

Synthetic patches are useful because they allow controlled experiments. Future work could create a larger synthetic patch suite with specific categories of changes, such as comment-only changes, formatting-only changes, performance improvements, complexity increases, deleted functionality, bypassed logic, added warnings, and runtime regressions.

This would make it possible to test whether the framework responds correctly to known patch patterns. It would also help validate edge cases, such as patches that reduce complexity by removing functionality or patches that compile successfully while skipping important behaviour.

12. Final Future Work Summary

Future work should therefore focus on turning PatchImpact from a thesis framework into a broader patch evaluation platform. The most important directions are support for more programming languages, larger datasets, better handling of large projects, more precise patch targeting, richer metrics, stronger dynamic analysis, improved heuristic calibration, and integration with real development workflows.

The foundation built in this thesis is suitable for these extensions because the framework already separates measurement, comparison, scoring, and reporting. This modular design means that new metrics, new languages, new analysis tools, and new scoring models can be added without changing the central idea of the work. The central idea remains that software patches should be evaluated not only by whether they change code, but by how they affect the structure, quality, and behaviour of the software system.

References

Ai, J., Guo, H., & Wong, W. E. (2020). What Ruined Your Cake: Impacts of Code Modifications on Bug Distribution. *IEEE Access*, *8*, 84020–84036. https://doi.org/10.1109/access.2020.2984179

Al-Bataineh, O. I., & Moonen, L. (2022). Towards Extending the Range of Bugs That Automated Program Repair Can Handle. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2211.03911

Alenezi, M. (2016). Software Architecture Quality Measurement Stability and Understandability. *International Journal of Advanced Computer Science and Applications*, *7*(7). https://doi.org/10.14569/ijacsa.2016.070775

AlOmar, E. A. (2025). An Empirical Study on the Impact of Code Duplication-aware Refactoring Practices on Quality Metrics. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2502.04073

Alqadi, B. S., & Maletic, J. I. (2020). *Slice-Based Cognitive Complexity Metrics for Defect Prediction*. 411–422. https://doi.org/10.1109/saner48275.2020.9054836

Attie, P. C., Obeidat, A., Oh, N., & Yelle, I. (2024). Code Documentation and Analysis to Secure Software Development. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2407.11934

Avery, J. (2017). The Application of Deception to Software Security Patching. *Purdue E-Pubs (Purdue University System)*. https://docs.lib.purdue.edu/dissertations/AAI10615541

Barrera, A. G. de la, Moraga, M. Á., Polo, M., & Guzmán, I. G. de. (2019). ProFit – Performing Dynamic Analysis of Software Systems. In *Communications in computer and information science* (pp. 255–262). Springer Science+Business Media. https://doi.org/10.1007/978-3-030-29238-6_18

Benaroch, M., & Lyytinen, K. (2022). How Much Does Software Complexity Matter for Maintenance Productivity? The Link Between Team Instability and Diversity. *IEEE Transactions on Software Engineering*, *49*(4), 2459–2475. https://doi.org/10.1109/tse.2022.3222119

Bernardi, G., Francalanza, A., Peressotti, M., & Mousavi, M. R. (2025). Software is infrastructure: failures, successes, costs, and the case for formal verification. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2506.13821

Bilokon, P., Lucuta, M., & Shermer, E. (2023). Semi-static Conditions in Low-latency C for High Frequency Trading: Better than Branch Prediction Hints. *SSRN Electronic Journal*. https://doi.org/10.2139/ssrn.4553439

Birihanu, E. (2018). Analysis of Software Quality Using Software Metrics. *International Journal on Computational Science & Applications*, *8*, 11–20. https://doi.org/10.5121/ijcsa.2018.8502

Bogner, J., & Verdecchia, R. (2025). ACM SIGSOFT SEN Empirical Software

Engineering: Introducing Our New Regular Column. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2510.02007

Böhme, M., Soremekun, E., Chattopadhyay, S., Ugherughe, E., & Zeller, A. (2017). *Where is the bug and how is it fixed? an experiment with practitioners*. 117–128. https://doi.org/10.1145/3106237.3106255

Brunner, T., Pataki, N., & Porkoláb, Z. (2016). Backward Compatibility Violations and their Detection in C++ Legacy Code using Static Analysis. *Acta Electrotechnica et Informatica*, *16*(2), 12–19. https://doi.org/10.15546/aeei-2016-0009

Chen, Z., & Jiang, L. (2024). Evaluating Software Development Agents: Patch Patterns, Code Quality, and Issue Complexity in Real-World GitHub Scenarios. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2410.12468

Chen, Z., & Jiang, L. (2025). *Evaluating Software Development Agents: Patch Patterns, Code Quality, and Issue Complexity in Real-World GitHub Scenarios*. 657–668. https://doi.org/10.1109/saner64311.2025.00068

Couto, M., Pereira, R., Ribeiro, F., Rua, R., & Saraiva, J. (2017). *Towards a Green Ranking for Programming Languages*. 1–8. https://doi.org/10.1145/3125374.3125382

Denisov-Blanch, Y., Ciobanu, I., Obstbaum, S., & Kosiński, M. (2024). Predicting Expert Evaluations in Software Code Reviews. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2409.15152

Dong, Y., Wu, M., Zhang, L., Yin, W., Wu, M., & Li, H. (2020). Priority Measurement of Patches for Program Repair Based on Semantic Distance. *Symmetry*, *12*(12), 2102–2102. https://doi.org/10.3390/sym12122102

Etemadi, K., Sharma, A., Madeiral, F., & Monperrus, M. (2023). Augmenting Diffs With Runtime Information. *IEEE Transactions on Software Engineering*, *49*(11), 4988–5007. https://doi.org/10.1109/tse.2023.3324258

Fei, Z., Ge, J., Li, C., Wang, T., Li, Y., Zhang, H., Huang, L., & Luo, B. (2024). Patch Correctness Assessment: A Survey. *ACM Transactions on Software Engineering and Methodology*, *34*(2), 1–50. https://doi.org/10.1145/3702972

Flater, D., Black, P. E., Fong, E., Kacker, R. N., Okun, V., Wong, S. E., & Kühn, R. (2016). *A Rational Foundation for Software Metrology*. https://doi.org/10.6028/nist.ir.8101

Furtado, V. R., Vignando, H., Fran a, V., & OliveiraJr, E. (2019). Comparing Approaches for Quality Evaluation of Software Engineering Experiments: An Empirical Study on Software Product Line Experiments. *Journal of Computer Science*, *15*(10), 1396–1429. https://doi.org/10.3844/jcssp.2019.1396.1429

Gamage, N. B. J. (2022). The risk factors affecting to the software quality failures in Sri Lankan Software industry. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.1705.09822

Gao, X., Noller, Y., & Roychoudhury, A. (2022). Program Repair. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2211.12787

Gao, X., Roychoudhury, A., Chin, W. N., Sergey, I., & Rinard, M. C. (2021). *OVERFITTING IN PROGRAM REPAIR AND SYNTHESIS*.

Gehlot, N., & Kaur, J. (2015). Dynamic inheritance coupling metric-design and analysis for assessing reusability. *International Journal of Software Engineering Technology and Applications*, *1*(1), 118–118. https://doi.org/10.1504/ijseta.2015.067536

Ghanbari, A. (2019). *Toward Practical Automatic Program Repair*. 1262–1264. https://doi.org/10.1109/ase.2019.00156

Gil, J. A., & Lalouche, G. (2016). When do Software Complexity Metrics Mean Nothing? – When Examined out of Context. *The Journal of Object Technology*, *15*(1). https://doi.org/10.5381/jot.2016.15.5.a2

Giordano, G., Festa, G., Catolino, G., Palomba, F., Ferrucci, F., & Gravino, C. (2023). On the adoption and effects of source code reuse on defect proneness and maintenance effort. *Empirical Software Engineering*, *29*(1). https://doi.org/10.1007/s10664-023-10408-6

Girka, T. (2018). Differential program semantics. *HAL (Le Centre Pour La Communication Scientifique Directe)*. https://hal.inria.fr/tel-01890508

Gomes dos Anjos, E., & Zenha-Rela, M. A. da C. (2017). *Assessing Maintainability in Software Architectures: The Elusive Socio-Technical Dimensions of Maintainability*.

Górski, J., & Kamiński, M. (2016). Towards automation of IT systems repairs. *Software Quality Journal*, *26*(1), 67–96. https://doi.org/10.1007/s11219-016-9335-5

Grazia, L. D., Bredl, P., & Pradel, M. (2022). DiffSearch: A Scalable and Precise Search Engine for Code Changes. *IEEE Transactions on Software Engineering*, *49*(4), 2366–2380. https://doi.org/10.1109/tse.2022.3218859

Grazia, L. D., & Pradel, M. (2022). Code Search: A Survey of Techniques for Finding Code [Review of *Code Search: A Survey of Techniques for Finding Code*]. *ACM Computing Surveys*, *55*(11), 1–31. Association for Computing Machinery. https://doi.org/10.1145/3565971

Herbold, S., Knieke, C., Rausch, A., & Schindler, C. (2025). Neurosymbolic Architectural Reasoning: Towards Formal Analysis through Neural Software Architecture Inference. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2503.16262

Herve, F. D. G. de. (2017). Reverse-engineering of binaries in a single execution : a lightweight function-grained dynamic analysis. *HAL (Le Centre Pour La Communication Scientifique Directe)*. https://tel.archives-ouvertes.fr/tel-01836311

Islam, C., Prokhorenko, V., & Babar, M. A. (2023). Runtime software patching: Taxonomy, survey and future directions. *Journal of Systems and Software*, *200*, 111652–111652. https://doi.org/10.1016/j.jss.2023.111652

Islam, M. R., & Sandborn, P. (2023). Demonstration of a Response Time Based Remaining Useful Life (RUL) Prediction for Software Systems. *Journal of Prognostics and Health Management*, *3*(1), 9–36. https://doi.org/10.22215/jphm.v3i1.3641

Ji, T., Wu, Y., Wang, C., Zhang, X., & Wang, Z. (2018). *The Coming Era of AlphaHacking?: A Survey of Automatic Software Vulnerability Detection, Exploitation and Patching Techniques*. https://doi.org/10.1109/dsc.2018.00017

Jiang, Y., Liu, H., Niu, N., Zhang, L., & Hu, Y. (2021). *Extracting Concise Bug-Fixing Patches from Human-Written Patches in Version Control Systems*. 686–698. https://doi.org/10.1109/icse43902.2021.00069

Karanikiotis, T., & Symeonidis, A. L. (2024). Towards Understanding the Impact of Code Modifications on Software Quality Metrics. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2404.03953

Kastner, D. L., Mauborgne, L., Wilhelm, S., Mallon, C., & Ferdinand, C. (2022). Static Data and Control Coupling Analysis. *HAL (Le Centre Pour La Communication Scientifique Directe)*. https://hal.archives-ouvertes.fr/hal-03694546

Kastner, D. L., Pister, M., & Ferdinand, C. (2022). Obtaining DO-178C Certification Credits by Static Program Analysis. *HAL (Le Centre Pour La Communication Scientifique Directe)*. https://hal.archives-ouvertes.fr/hal-03694553

Khleel, N. A. A., & Nehéz, K. (2021). Comprehensive Study on Machine Learning Techniques for Software Bug Prediction. *International Journal of Advanced Computer Science and Applications*, *12*(8). https://doi.org/10.14569/ijacsa.2021.0120884

Kim, J.-H., Park, J., & Yoo, S. (2023). *The Inversive Relationship Between Bugs and Patches: An Empirical Study*. 314–323. https://doi.org/10.1109/icstw58534.2023.00062

Koyuncu, A., Liu, K., Bissyandé, T. F., Kim, D.-S., Klein, J., Monperrus, M., Traon, Y. L., Koyuncu, A., Liu, K., Bissyandé, T. F., Kim, D.-S., Klein, J., Monperrus, M., & Traon, Y. L. (2020). FixMiner: Mining relevant fix patterns for automated program repair. *Empirical Software Engineering*, *25*(3), 1980–2024. https://doi.org/10.1007/s10664-019-09780-z

Laaber, C., Basmaci, M., & Salza, P. (2021). Predicting unstable software benchmarks using static source code features. *Empirical Software Engineering*, *26*(6). https://doi.org/10.1007/s10664-021-09996-y

Laaber, C., Yue, T., & Ali, S. (2024). Evaluating Search-Based Software Microbenchmark Prioritization. *IEEE Transactions on Software Engineering*, *50*(7), 1687–1703. https://doi.org/10.1109/tse.2024.3380836

Latimer, M. (2017). Trace-weighted binary comparison for software update management. *IDEALS (University of Illinois Urbana-Champaign)*.

http://hdl.handle.net/2142/99389

Lazreg, S., Cordy, M., Collet, P., Heymans, P., & Mosser, S. (2019). *Multifaceted Automated Analyses for Variability-Intensive Embedded Systems*. *15*, 854–865. https://doi.org/10.1109/icse.2019.00092

Le-Cong, T., Luong, D.-M., Le, X.-B. D., Lo, D., Tran, N.-H., Quang-Huy, B., & Huynh, Q. (2023). Invalidator: Automated Patch Correctness Assessment via Semantic and Syntactic Reasoning. *IEEE Transactions on Software Engineering*, 1–20. https://doi.org/10.1109/tse.2023.3255177

Li, F., & Paxson, V. (2017). *A Large-Scale Empirical Study of Security Patches*. 2201–2215. https://doi.org/10.1145/3133956.3134072

Li, Y., Shezan, F. H., wei, B., Wang, G., & Tian, Y. (2025). SoK: Towards Effective Automated Vulnerability Repair. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2501.18820

Liang, J., Hou, Y., Zhou, S., Chen, J., Xiong, Y., & Huang, G. (2019). *How to Explain a Patch: An Empirical Study of Patch Explanations in Open Source Projects*. https://doi.org/10.1109/issre.2019.00016

Liu, K., Kim, D., Koyuncu, A., Li, L., Bissyandé, T. F., & Traon, Y. L. (2018). *A Closer Look at Real-World Patches*. 275–286. https://doi.org/10.1109/icsme.2018.00037

Liu, K., Li, L., Koyuncu, A., Kim, D., Liu, Z., Klein, J., & Bissyandé, T. F. (2020). A critical review on the evaluation of automated program repair systems [Review of *A critical review on the evaluation of automated program repair systems*]. *Journal of Systems and Software*, *171*, 110817–110817. Elsevier BV. https://doi.org/10.1016/j.jss.2020.110817

Longo, R., Pintore, F., Rinaldo, G., & Sala, M. (2017). *On the security of the blockchain BIX protocol and certificates*. *1561*, 1–16. https://doi.org/10.23919/cycon.2017.8240338

Lukić, R. (2022). OPERATING COSTS OF TRADE IN SERBIA. *Southeast European Review of Business and Economics*, *3*(1), 26–45. https://doi.org/10.20544/serbe.05.01.22.p02

Mann, S., Pathak, N., Sharma, N., Kumar, R., Porwal, R., Sharma, S. K., & Aung, S. M. Y. (2022). Study of Energy-Efficient Optimization Techniques for High-Level Homogeneous Resource Management. *Wireless Communications and Mobile Computing*, *2022*(1). https://doi.org/10.1155/2022/1953510

Martínez, M., Durieux, T., Sommerard, R., Xuan, J., & Monperrus, M. (2016). Automatic repair of real bugs in java: a large-scale experiment on the defects4j dataset. *Empirical Software Engineering*, *22*(4), 1936–1964. https://doi.org/10.1007/s10664-016-9470-4

Mastandrea, V. (2017). Analysis of synchronisation patterns in active objects based on behavioural types. *HAL (Le Centre Pour La Communication Scientifique Directe)*. https://hal.science/tel-01651649

Merlo, A., Ruggia, A., Sciolla, L., & Verderame, L. (2021). You Shall not Repackage! Demystifying Anti-Repackaging on Android. *Computers & Security*, *103*, 102181–102181. https://doi.org/10.1016/j.cose.2021.102181

Monperrus, M. (2018). The Living Review on Automated Program Repair. *HAL (Le Centre Pour La Communication Scientifique Directe)*. https://hal.archives-ouvertes.fr/hal-01956501

Morice, V. (2022). A High Level Model Based on Hardware Dependencies for The Development of Embedded Software on Microcontrollers. *HAL (Le Centre Pour La Communication Scientifique Directe)*. https://theses.hal.science/tel-04076218

Muench, M. (2019). Dynamic binary firmware analysis : challenges & solutions. *HAL (Le Centre Pour La Communication Scientifique Directe)*. https://theses.hal.science/tel-03143960

Oliveira, D., Santos, R., Oliveira, B. de, Monperrus, M., Castor, F., & Madeiral, F. (2024). Understanding Code Understandability Improvements in Code Reviews. *IEEE Transactions on Software Engineering*, *51*(1), 14–37. https://doi.org/10.1109/tse.2024.3453783

Omri, S., Montag, P., & Sinz, C. (2018). Static Analysis and Code Complexity Metrics as Early Indicators of Software Defects. *Journal of Software Engineering and Applications*, *11*(4), 153–166. https://doi.org/10.4236/jsea.2018.114010

Ponta, S. E., Plate, H., & Sabetta, A. (2018). *Beyond Metadata: Code-Centric and Usage-Based Analysis of Known Vulnerabilities in Open-Source Software*. https://doi.org/10.1109/icsme.2018.00054

Presler-Marshall, K., Heckman, S., Stolee, K., Bottomley, L., & Lynch, C. (2022). *Supporting Computer Science Education Through Automation and Surveys*.

Ramsauer, R., Lohmann, D., & Mauerer, W. (2016). Observing Custom Software Modifications: A Quantitative Approach of Tracking the Evolution of Patch Stacks. *arXiv (Cornell University)*, 4. https://doi.org/10.48550/arxiv.1607.00905

Reis, S., Abreu, R., & Cruz, L. (2021). Fixing vulnerabilities potentially hinders maintainability. *Empirical Software Engineering*, *26*(6). https://doi.org/10.1007/s10664-021-10019-z

Sejfia, A., Zhao, Y., & Medvidović, N. (2021). *Identifying casualty changes in software patches*. 304–315. https://doi.org/10.1145/3468264.3468624

Setiadi, D., Sumitra, T., Karim, A., & Ritzkal, R. (2024). Software Quality Measurement Analysis on Academic Information Systems. *Ingénierie Des Systèmes d Information*, *29*(4), 1453–1460. https://doi.org/10.18280/isi.290418

Shah, Y. (2017). Brief Review of Various Metrics to Achieve Reusability and Changeability of a Software. *International Journal for Research in Applied Science and Engineering Technology*, 309–313. https://doi.org/10.22214/ijraset.2017.4057

Shirazi, M. T. (2019). Analysis of obfuscation transformations on binary code. *HAL (Le Centre Pour La Communication Scientifique Directe)*. https://theses.hal.science/tel-02907117

Smith, E. K., Barr, E. T., Goues, C. L., & Brun, Y. (2015). *Is the cure worse than the disease? overfitting in automated program repair*. https://doi.org/10.1145/2786805.2786825

Sobhy, D., Bahsoon, R., Minku, L. L., & Kazman, R. (2021). Evaluation of Software Architectures under Uncertainty. *ACM Transactions on Software Engineering and Methodology*, *30*(4), 1–50. https://doi.org/10.1145/3464305

Sobreira, V., Durieux, T., Madeiral, F., Monperrus, M., & Maia, M. de A. (2018). *Dissection of a bug dataset: Anatomy of 395 patches from Defects4J*. https://doi.org/10.1109/saner.2018.8330203

Souza, R. F. de, Chávez, C., & Bittencourt, R. A. (2015). Rapid Releases and Patch Backouts: A Software Analytics Approach. *IEEE Software*, *32*(2), 89–96. https://doi.org/10.1109/ms.2015.30

Souza, R. R., Chavez, C. F. G., & Bittencourt, R. A. (2015). Patch rejection in Firefox: negative reviews, backouts, and issue reopening. *Journal of Software Engineering Research and Development*, *3*(1). https://doi.org/10.1186/s40411-015-0024-z

Spring, J., & Illari, P. (2018). Building General Knowledge of Mechanisms in Information Security. *Philosophy & Technology*, *32*(4), 627–659. https://doi.org/10.1007/s13347-018-0329-z

Tian, H., Liu, K., Kaboreé, A. K., Koyuncu, A., Li, L., Klein, J., & Bissyandé, T. F. (2022). Evaluating Representation Learning of Code Changes for Predicting Patch Correctness in Program Repair. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2008.02944

Trautsch, A., Erbel, J., Herbold, S., & Grabowski, J. (2023). What really changes when developers intend to improve their source code: a commit-level study of static metric value and static analysis warning changes. *Empirical Software Engineering*, *28*(2). https://doi.org/10.1007/s10664-022-10257-9

Tufano, M., Watson, C., Bavota, G., Penta, M. D., White, M., & Poshyvanyk, D. (2019). *Learning How to Mutate Source Code from Bug-Fixes*. https://doi.org/10.1109/icsme.2019.00046

Vida, R. F. (2017). Improving Program Comprehension through Dynamic Code Analysis. *Studia Universitatis Babeș-Bolyai Informatica*, *62*(2), 69–82. https://doi.org/10.24193/subbi.2017.2.06

Voas, J., & Kuhn, R. B. (2017). What Happened to Software Metrics? *Computer*, *50*(5), 88–98. https://doi.org/10.1109/mc.2017.144

Wang, G., Chattopadhyay, S., Gotovchits, I., Mitra, T., & Roychoudhury, A. (2019). oo7: Low-Overhead Defense Against Spectre Attacks via Program

Analysis. *IEEE Transactions on Software Engineering*, *47*(11), 2504–2519. https://doi.org/10.1109/tse.2019.2953709

Wang, S., Wen, M., Chen, L., Yi, X., & Mao, X. (2019). *How Different Is It Between Machine-Generated and Developer-Provided Patches? : An Empirical Study on the Correct Patches Generated by Automated Program Repair Techniques*. https://doi.org/10.1109/esem.2019.8870172

Whalen, M. W., Person, S., Rungta, N., Staats, M., & Grijincu, D. (2015). A Flexible and Non-intrusive Approach for Computing Complex Structural Coverage Metrics. *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, 506–516. https://doi.org/10.1109/icse.2015.68

Wu, W.-C., Yan, Y., Egilsson, H. D., Park, D., Chan, S., Hauser, C., & Wang, W. (2024). Is This the Same Code? A Comprehensive Study of Decompilation Techniques for WebAssembly Binaries. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2411.02278

Xiao, Y., Yang, C., Wang, B., & Xiong, Y. (2024). Accelerating Patch Validation for Program Repair With Interception-Based Execution Scheduling. *IEEE Transactions on Software Engineering*, *50*(3), 618–635. https://doi.org/10.1109/tse.2024.3359969

Yang, Z., Gao, C., Guo, Z., Li, Z., Liu, K., Xia, X., & Zhou, Y. (2024). A Survey on Modern Code Review: Progresses, Challenges and Opportunities. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2405.18216

Ye, H., Martínez, M., & Monperrus, M. (2021). Automated patch assessment for program repair at scale. *Empirical Software Engineering*, *26*(2). https://doi.org/10.1007/s10664-020-09920-w

Yeltsin, I., Araújo, A. A., Dantas, A., Soares, P., Saraiva, R., & Souza, J. (2025). On the "fairness" of search-based software engineering: Investigating the capability of a scalarizing-based function to control the software metrics' influence on optimization process. *Journal of Software Engineering Research and Development*, *13*(1). https://doi.org/10.5753/jserd.2025.3906

Yue, R., Meng, N., & Wang, Q. (2017). *A Characterization Study of Repeated Bug Fixes*. 422–432. https://doi.org/10.1109/icsme.2017.16

Zhao, J. (2021). *Some Size and Structure Metrics for Quantum Software*. 22–27. https://doi.org/10.1109/q-se52541.2021.00012

Zhong, H., & Su, Z. (2015, May 1). An Empirical Study on Real Bug Fixes. *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. https://doi.org/10.1109/icse.2015.101

Zia, G. A. J., Hanke, T., Skrotzki, B., Völker, C., & Bayerlein, B. (2024). Enhancing Reproducibility in Precipitate Analysis: A FAIR Approach with Automated Dark-Field Transmission Electron Microscope Image Processing. *Integrating Materials and Manufacturing Innovation*, *13*(1), 257–271.

https://doi.org/10.1007/s40192-023-00331-5

Zibaeirad, A., & Vieira, M. (2024). VulnLLMEval: A Framework for Evaluating Large Language Models in Software Vulnerability Detection and Patching. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2409.10756

Liang, J., Hou, Y., Zhou, S., Chen, J., Xiong, Y., & Huang, G. (2019). How to explain a patch: An empirical study of patch explanations in open source projects. 2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE). https://doi.org/10.1109/ISSRE.2019.00016

Martínez, M., Durieux, T., Sommerard, R., Xuan, J., & Monperrus, M. (2017). Automatic repair of real bugs in Java: A large-scale experiment on the Defects4J dataset. Empirical Software Engineering, 22(4), 1936-1964.

McCabe, T. J. (1976). A complexity measure. IEEE Transactions on Software Engineering, SE-2(4), 308-320.

Smith, E. K., Barr, E. T., Le Goues, C., & Brun, Y. (2015). Is the cure worse than the disease? Overfitting in automated program repair. Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. https://doi.org/10.1145/2786805.2786825

Souza, R. F. de, Chávez, C., & Bittencourt, R. A. (2015). Rapid releases and patch backouts: A software analytics approach. IEEE Software, 32(2), 89-96. https://doi.org/10.1109/MS.2015.30

Trautsch, A., Erbel, J., Herbold, S., & Grabowski, J. (2023). What really changes when developers intend to improve their source code: A commit-level study of static metric value and static analysis warning changes. Empirical Software Engineering, 28(2). https://doi.org/10.1007/s10664-022-10257-9

Appendices

These appendices provide supporting material for reproducibility and implementation transparency. The full PatchImpact implementation is available in the project repository, while selected code excerpts, execution commands, representative outputs, dataset information, and roadmap notes are included here to document how the framework was implemented and used.

Appendix A: PatchImpact Source Code Repository

The full source code of the PatchImpact framework is available in the project repository:

https://github.com/AthanasiosKarathanasis/PatchImpact

The repository contains the C++17 implementation of the framework, including the source files, header files, build configuration, and usage instructions. It includes the main analysis modules for structural impact, quality impact, behavioural measurement, CSV export, report generation, and overall patch-impact scoring.

The purpose of providing the repository is to support transparency and reproducibility. Instead of including the full implementation directly in the thesis body, selected implementation excerpts are included in Appendix C, while the complete framework is made available through the repository. This keeps the thesis readable while still making the implementation accessible for inspection, reuse, and future extension.

A.1 Repository Contents

Item	Purpose
README.md	Provides a short project description, build commands, usage examples, and thesis context.
CMakeLists.txt and config.h.in	Define the CMake build configuration and version-related configuration.
include/	Contains the public header files for the framework modules.
src/	Contains the implementation files for the command-line application and analysis modules.

A.2 Reproducibility Role

The repository supports reproducibility by allowing another reader to inspect the implementation, build the framework, and compare the source structure with the implementation description in the thesis. It also makes the work easier to extend after submission, for example by adding new metrics, new output formats, or support for additional programming languages.

Appendix B: Framework Installation, Execution, and Reproducibility Guide

This appendix summarizes how PatchImpact can be installed, built, and executed. It is a shortened and formalized version of the working framework notes used during development.

B.1 Purpose of the Framework

PatchImpact is a command-line framework developed for this thesis to support the evaluation of software patch impact in C and C++ projects. Its purpose is to collect measurable evidence about how a patch affects source code structure, static code quality, executable behaviour, and overall patch risk. The framework is a research prototype, not a production-grade industrial tool. Its main role is to support controlled thesis experiments by comparing original and patched software artefacts in a repeatable way.

B.2 Development Environment

Category	**Environment used in the thesis**
Operating system	Arch Linux
Desktop environment	XFCE
Programming language	C++17
Build system	CMake with Ninja
Compiler toolchain	GCC and G++
External analysis tools	pmccabe, Cppcheck, GNU time, diffutils

The framework may be portable to other Linux distributions, but reproducibility is strongest when using a similar Linux environment with equivalent tool versions installed.

B.3 Required Software

Tool	Role in the framework
GCC and G++	Compile the framework and support C and C++ project builds.
CMake	Configure the build system.
Ninja	Provide a fast and reproducible build backend.
Cppcheck	Collect static analysis findings such as warnings, style issues, and informational messages.
pmccabe	Estimate cyclomatic complexity and function counts for C and C++ source files.
GNU time	Collect runtime measurements such as CPU time, memory usage, and exit status for executables.
diffutils	Support line-level difference analysis.

B.4 Example Installation Commands on Arch Linux

```
sudo pacman -Syu
sudo pacman -S gcc cmake ninja cppcheck diffutils git make
```

The pmccabe tool was installed from source in the thesis environment when it was not available through the normal package workflow:

```
cd ~/Downloads
git clone https://gitlab.com/pmccabe/pmccabe.git
cd pmccabe
make
sudo make install
whereis pmccabe
```

B.5 Building PatchImpact

```
cd patchimpact_v41
mkdir -p build
cd build
cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
-G Ninja ..
cmake --build . --target all
```

A successful build produces the PatchImpact executable used in the experiments. In the thesis environment, the executable name was patchimpact_v41.

B.6 Main Execution Modes

Mode	Command pattern	Purpose
Single folder source analysis	./patchimpact_v41 <source_folder>	Scans a source folder and reports metrics for supported C and C++ files.
Folder comparison	./patchimpact_v41 <original_folder> <patched_folder>	Compares original and patched folders and produces structural, quality, and overall assessment results.
Build analysis	./patchimpact_v41 -ba <original_folder> <patched_folder>	Compares build status and build exit codes for two project folders.
Dynamic measurement	./patchimpact_v41 -dm <original_executable> <patched_executable>	Compares executable-level runtime metrics, binary size, and exit status.
Individual assessment	./patchimpact_v41 -ia <folder_or_file>	Inspects a single source file, executable, or folder.
Mass assessment	./patchimpact_v41 -ma <folder>	Performs pairwise directional comparison of compatible artefacts in a folder.

B.7 Report Generation and CSV Export

PatchImpact supports text report generation using the report modifier:

./patchimpact_v41 -rep <output_file> <existing_command>

Examples:

./patchimpact_v41 -rep report.txt original_folder patched_folder
./patchimpact_v41 -rep dynamic_report.txt -dm original_executable patched_executable
./patchimpact_v41 -rep mass_report.txt -ma project_folder

PatchImpact also supports structured CSV export:

./patchimpact_v41 -csv <output_file> <existing_command>

Examples:

./patchimpact_v41 -csv individual.csv -ia project_folder
./patchimpact_v41 -csv mass.csv -ma project_folder

The report files preserve human-readable terminal output, while CSV files provide structured data for spreadsheet or statistical analysis.

B.8 Reproducibility Notes

To reproduce an experiment, the following information should be recorded: operating system, compiler version, CMake version, Ninja version, Cppcheck version, pmccabe installation, PatchImpact version, input projects, command-line arguments, generated reports, and generated CSV files. This information makes it possible for another researcher to understand the experimental setup and repeat the measurements under similar conditions.

Appendix C: Selected Implementation Excerpts

The complete implementation is available in the public repository listed in Appendix A. For readability, this appendix includes selected simplified excerpts that illustrate the main structure of the framework. These excerpts show the core implementation ideas without reproducing every source file in the thesis.

C.1 Source File Collection

The framework recursively scans project folders and collects supported C and C++ source and header files. Unsupported files and build artefacts are excluded so that the analysis remains focused on relevant software artefacts.

```
// Simplified source file collection logic

for (const auto& entry : std::filesystem::recursive_directory_iterator(rootPath))
{
if (!entry.is_regular_file()) {
continue;
}

std::string extension = entry.path().extension().string();

if (extension == ".c" ||
extension == ".cpp" ||
extension == ".h" ||
extension == ".hpp") {
sourceFiles.push_back(entry.path());
}
}
```

C.2 Dynamic Measurement Comparison

Executable-level comparison is separated from source-folder comparison. This allows the framework to measure runtime behaviour when suitable executables are available, without forcing every folder comparison to include executable analysis.

```
// Simplified dynamic measurement workflow

DynamicMetrics originalMetrics = measureExecutable(originalExecutable);
DynamicMetrics patchedMetrics = measureExecutable(patchedExecutable);

DynamicComparison comparison;
comparison.elapsedChange =
patchedMetrics.elapsedSeconds - originalMetrics.elapsedSeconds;

comparison.memoryChange =
patchedMetrics.maximumResidentMemoryKb -
originalMetrics.maximumResidentMemoryKb;

comparison.binarySizeChange =
patchedMetrics.binarySizeBytes - originalMetrics.binarySizeBytes;

comparison.exitStatusChanged =
patchedMetrics.exitStatus != originalMetrics.exitStatus;
```

C.3 Command Mode Dispatch

PatchImpact uses command-line dispatch to select the correct analysis mode. This design supports both small manual experiments and larger dataset-based workflows.

```
// Simplified command mode dispatch

int PatchImpactApp::run(int argc, char* argv[]) {
CommandLineOptions options = parseArguments(argc, argv);

if (options.mode == Mode::IndividualAssessment) {
return individualAssessmentAnalyzer.run(options.inputPath);
}

if (options.mode == Mode::MassAssessment) {
return massAssessmentAnalyzer.run(options.inputPath);
}

if (options.mode == Mode::DynamicMeasurement) {
return dynamicAnalyzer.compare(
options.originalExecutable,
options.patchedExecutable
);
}

if (options.mode == Mode::BuildAnalysis) {
```

```
return buildAnalyzer.compare(
options.originalFolder,
options.patchedFolder
);
}

if (options.mode == Mode::FolderComparison) {
return compareFolders(
options.originalFolder,
options.patchedFolder
);
}

return showHelp();
}
```

C.4 Metric Change Calculation

Metric comparison is based on storing original and patched values, calculating absolute change, and calculating percentage change only when the original value is nonzero. This avoids invalid division and allows zero-value edge cases to be interpreted separately.

```
// Simplified metric comparison

MetricChange calculateChange(double originalValue, double patchedValue) {
MetricChange change;
change.originalValue = originalValue;
change.patchedValue = patchedValue;
change.absoluteChange = patchedValue - originalValue;

if (originalValue != 0.0) {
change.percentageChange =
((patchedValue - originalValue) / originalValue) * 100.0;
change.percentageAvailable = true;
} else {
change.percentageChange = 0.0;
change.percentageAvailable = false;
}

return change;
}
```

C.5 CSV Export Logic

CSV export allows framework results to be reused in spreadsheet and analysis tools. This was important for the results chapter because it separated structured experimental data from raw terminal output.

```
// Simplified CSV export logic

void CsvExportManager::writeSourcePairRow(
const SourcePairResult& result
) {
csv « result.originalPath « ","
« result.patchedPath « ","
« result.originalComplexity « ","
« result.patchedComplexity « ","
« result.complexityChange « ","
« result.originalFunctions « ","
« result.patchedFunctions « ","
« result.functionChange « ","
« result.originalSloc « ","
« result.patchedSloc « ","
« result.slocChange « ","
« result.similarityPercent « ","
« result.structuralScore « ","
« result.qualityScore « "
";
}
```

Appendix D: Example Framework Outputs

This appendix provides representative examples of the framework output used in the thesis. Full raw reports and full CSV files are not reproduced in full because they would reduce readability. Instead, key summaries and representative structures are included.

D.1 Complete Berry Folder-Comparison Results

Patch pair	Overall score	Structural score	Quality score	Grade	Risk level	Changed files
Berry 1	100.0000	100.0000	100	A	Low	1
Berry 2	99.9993	99.9985	100	A	Low	1
Berry 3	97.4992	99.9984	95	A	Low	1
Berry 4	99.9895	99.9790	100	A	Low	1
Berry 5	99.9922	99.9845	100	A	Low	1

Berry 3 is included as an especially useful representative case because it shows

that structural impact and quality impact can differ. The structural score remained very high, while the quality score decreased because the static analysis findings changed.

D.2 Mini Mass Assessment Summary

Metric	Value
Total rows	28,436
Source-pair rows	28,056
Executable-pair rows	380
Average structural score	94.46 / 100
Average quality score	95.75 / 100

The mini mass assessment was used for larger-scale metric generation and CSV-based analysis. These rows are pairwise directional artefact comparisons, not independent real-world patches.

D.3 CSV Header Example

The following simplified CSV header illustrates the structure of the mass assessment output:

assessment_type, original_path, patched_path,
original_complexity, patched_complexity, complexity_change,
original_functions, patched_functions, function_change,
original_sloc, patched_sloc, sloc_change,
similarity_percent, maintainability_change, cppcheck_change,
structural_score, quality_score,
original_elapsed, patched_elapsed,
original_cpu_total, patched_cpu_total,
original_memory_kb, patched_memory_kb,
original_binary_bytes, patched_binary_bytes,
original_exit_status, patched_exit_status,
elapsed_change_percent, cpu_change_percent,
memory_change_percent, binary_change_percent,
exit_status_changed, dynamic_score, grade, risk_level

Appendix E: Dataset and Source Repository Information

The empirical work used open-source C and C++ software artefacts as the basis for patch-impact evaluation. The main dataset source was BugsCpp, which provides buggy and fixed versions of C and C++ projects suitable for experimental software engineering work. The local dataset used in the thesis was prepared under the thesis dataset folder on the development machine.

E.1 Dataset Source and Project Repositories

Project or source	Repository or source link	Use in this thesis
PatchImpact framework	https://github.com/AthanasiosKarathanasis/PatchImpact	Basis/Repository for the thesis framework implementation.
BugsCpp dataset	https://github.com/Suresoft-GLaDOS/bugscpp	Main source for buggy and fixed C/C++ project versions.
Berry	https://github.com/berry-lang/berry	Used for complete scored folder-comparison results.
GNU Coreutils	https://github.com/coreutils/coreutils	Considered and attempted as a large real-world C project. It was not treated as a final scored result because the report did not reach the final scored evaluation section.
cpp-peglib	https://github.com/yhirose/cpp-peglib.git	Considered as part of the available BugsCpp project sources.
Cppcheck	https://github.com/danmar/cppcheck.git	Available in the dataset source list and also used as an external static analysis tool.

E.2 Dataset Interpretation

The final scored folder-comparison results used in the results chapter were based on the complete Berry patch-pair reports. Coreutils was useful as a large-project case because it showed the practical difficulty of analysing large C projects with many files and dependencies. However, because the Coreutils report did not produce a complete final scored summary, it was not included as a final scored result. The mini mass assessment was used as an exploratory dataset for larger-scale metric generation and CSV-based analysis.

E.3 Material Not Included in the Thesis Body

Full third-party project source code, the full BugsCpp dataset, and complete large CSV files are not included directly in the thesis document. Including them would make the document unnecessarily large and reduce readability. Instead, the thesis includes representative summaries and points to the relevant repositories and generated artefacts.

Appendix F: Development Roadmap and Edge-Case Considerations

This appendix summarizes the development roadmap and edge-case considerations that informed the design of PatchImpact. It is based on the framework planning notes used during implementation, rewritten here as formal supporting material.

F.1 Completed Development Stages

Version stage	Implemented capability
0.5	File matching.
0.6	Changed and unchanged file detection.
0.7	Diff added-line and removed-line analysis.
0.8	Cyclomatic complexity analysis using pmccabe.
0.9	Cppcheck integration.
0.10	Categorized Cppcheck findings, including errors, warnings, style, and information.
0.11	Function count using pmccabe.
0.12	Ignored generated and irrelevant folders such as build folders, CMake files, Git metadata, and IDE settings.
0.13 and 0.14	Rename and similar filename candidate detection with metrics for rename candidates.

F.2 Metric Expansion Considered During Development

Area	Possible or implemented metrics
Static metrics	Source lines of code, comment lines, blank lines, maintainability index, average complexity per function, maximum complexity function, complexity density, functions per file, file size statistics.
Dynamic metrics	Execution time, repeated runs, average runtime, peak memory usage, original versus patched runtime difference.
Patch similarity	Levenshtein distance, similarity percentage, edit distance, patch-size estimation, logical-distance estimation.
Visualisation and reporting	Graphviz diagrams, CSV export, spreadsheet import, optional future HTML reports.

F.3 Edge Cases Considered

Several edge cases were considered because metric-based patch evaluation can be misleading if reductions in complexity or warnings are caused by deleting functionality rather than improving it. These cases informed both the implementation and the discussion of limitations.

- A patch removes an entire file.
- A patch comments out an entire file.
- A patch removes problematic code instead of fixing it.
- A patch reduces complexity by removing functionality.
- A patch reduces warnings by deleting functionality.
- A patch compiles but bypasses behaviour.
- A patch compiles but performs no useful work.
- An empty patch is submitted.
- A file is renamed without meaningful changes.
- A large patch has minimal behavioural impact.
- A small patch has major behavioural impact.

F.4 Long-Term Architecture Direction

The long-term architecture remains compatible with the pipeline model used in the thesis: file discovery, file pairing, metric collection, metric comparison, heuristic evaluation, patch grade, and final report. This supports future extension because new metrics and tools can be added without changing the central idea of comparing original and patched artefacts through measurable evidence.